Collège Sciences et Technologies
UF Mathématiques et Interactions

Comparaison de Méta-Heuristiques pour le problème de partitionnement des graphes

Melissa AFTISSE
Gayathiri RAVENDIRANE
Anny WINGERS

ROAD M1
2025 – 2026

Outils logiciels pour l'optimisation: Méta-heuristiques

Table des matières

1 Introduction **5**

2 Définitions et fondements théoriques **6**
2.1 Formalisation du problème . . . 6
2.1.1 Instance . . . 6
2.1.2 Solution . . . 6
2.1.3 La fonction objectif . . . 6
2.1.4 La contrainte d'équité . . . 6
2.1.5 Exemple . . . 7
2.1.6 Des voisinages et un mouvement élémentaire . . . 8
2.2 Structures de données et outils de base . . . 9
2.2.1 Représentation du graphe . . . 9
2.2.2 Représentation d'une partition . . . 10
2.2.3 Les calculs incrémentaux . . . 11

3 Méthodes de résolution **12**
3.1 Méthodes exactes . . . 12
3.1.1 Énumération explicite . . . 12
3.2 Méthodes heuristiques locales . . . 12
3.2.1 Descente de gradient (algorithme glouton du gradient sans optimisation) . . . 12
3.2.2 Algorithme de Kernighan & Lin (KL) . . . 13
3.2.3 Gradient avec heuristique . . . 14
3.3 Méta-heuristiques . . . 15
3.3.1 Le recuit simulé . . . 15
3.3.2 Algorithme génétique simple (AGS) . . . 17

4 Expérimentations et résultats **21**
4.1 Méthodes exactes - Énumération explicite . . . 21
4.2 Méthodes heuristiques locales . . . 22
4.2.1 Algorithme du gradient (version récursive) . . . 22
4.2.2 Kernighan & Lin (KL) . . . 24
4.2.3 Gradient avec heuristique . . . 25
4.3 Le recuit simulé . . . 25
4.4 Algorithme génétique simple (AGS) . . . 27
4.5 Analyse des résultats de l'AGS . . . 27

5 Analyse globale et discussion **31**

6 Conclusion **32**

A Résultats détaillés pour les algorithmes **33**
A.1 L'algorithme d'énumération explicite . . . 33

Engagement de non plagiat

Nous, Melissa AFTISSE, Gayathiri RAVENDIRANE, Anny WINGERS, nous engageons fermement à éviter tout acte de plagiat dans ce rapport. En signant cet engagement :

- nous nous engageons à citer correctement toutes les sources utilisées dans nos travaux, en suivant les conventions de citation appropriées.
- nous nous engageons à produire un rapport original, évitant tout recours direct ou indirect au travail d'autrui sans attribution adéquate.
- nous nous engageons à respecter les droits d'auteur et la propriété intellectuelle, en obtenant les autorisations nécessaires pour utiliser et citer des oeuvres protégées par le droit d'auteur.

Nous comprenons que tout acte de plagiat peut entraîner des conséquences disciplinaires.

Fait à Talence le 17 avril 2026

Signatures

Melissa AFTISSE
Gayathiri RAVENDIRANE
Anny WINGERS

Engagement à l'égard de l'Intelligence Artificielle

Nous, Melissa AFTISSE, Gayathiri RAVENDIRANE, Anny WINGERS, nous engageons à ne pas utiliser d'Intelligence Artificielle générative de texte dans le cadre de la rédaction de ce rapport. Nous nous engageons à utiliser d'Intelligence Artificielle générative de code uniquement pour des besoins ponctuels en documentant clairement son utilisation dans le rapport.

Dans le cadre de ce rapport, certaines utilisations ponctuelles de l'Intelligence Artificielle ont été effectuées et sont documentées ci-dessous :

- Vérification grammaticale et orthographique du texte réalisée.
- L'exemple et l'illustration pour le problème.
- Optimisation des codes fournis (notamment pour l'algorithme du gradient et du recuit simulé).
- Assistance pour l'utilisation de la bibliothèque NetworkX dans le code.
- Aide à la mise en place d'une interface graphique (GUI).

Nous comprenons que toute violation de cet engagement peut entraîner des conséquences disciplinaires.

Fait à Talence le 17 avril 2026

Signatures

Melissa AFTISSE
Gayathiri RAVENDIRANE
Anny WINGERS

1 Introduction

Essentielle aux mathématiques discrètes et à l'informatique, la théorie des graphes offre un cadre d'abstraction pour modéliser et résoudre des problèmes réels et complexes. En représentant les informations sous forme de graphes (ensemble de sommets interconnectés par des arêtes), cette discipline permet d'optimiser des problèmes variés, notamment ceux en télécommunication, réseaux, biologie et bien d'autres. Les fondements de cette théorie remontent aux travaux de Léonard Euler, notamment à travers le célèbre problème des sept ponts de Königsberg, qui consistait à partir d'une ville : passer une et exactement une seule fois par chaque pont et revenir au point de départ.

L'étude menée ci-dessous se focalise sur l'un des problèmes majeurs de la théorie des graphes. Il s'agit du problème de partitionnement de graphes qui consiste à segmenter l'ensemble des sommets en un certain nombre entier p de sous ensembles représentant des classes à peu près équitables (de cardinalité sensiblement égale). L'enjeu majeur est d'assurer la contrainte d'équitabilité entre les classes tout en minimisant le poids total des arêtes inter-classes.

On s'intéresse à cette problématique particulièrement parce qu'elle est au coeur de nombreuses applications pratiques telles que : le calcul parallèle, la conception de circuits électroniques, l'optimisation des bases de données, etc. Concernant le calcul parallèle, par exemple, la technique de partitionnement est largement utilisée puisque l'on cherche à répartir une tâche massive (représentée sous forme de graphe) sur plusieurs processeurs ; une solution optimale assure l'équilibre en termes de charge de travail, réduit les échanges de données lents et garantit également une accélération maximale du traitement. Cette logique est également très importante dans la conception de réseaux électroniques en modélisant les composantes électroniques comme des sommets et leurs interconnexions par des arêtes. Un partitionnement optimal permet alors de regrouper de manière efficace les composants qui communiquent de manière intense sur une même zone du circuit. L'objectif est de réduire la longueur totale des fils conducteurs, ce qui diminue le temps de propagation du signal, la consommation d'énergie et la surchauffe du matériel également.

Face à l'explosion de la taille des réseaux à traiter, aux défis contemporains et à l'impossibilité de trouver des solutions optimales en un temps raisonnable, un recours aux métaheuristiques s'impose comme une alternative stratégique pour surmonter toutes ces limites. Reconnues pour leur efficacité, les métaheuristiques surpassent souvent les méthodes traditionnelles pour résoudre les problèmes de grande complexité. Grâce à leur polyvalence, elles s'adaptent à des problèmes différents issus de domaines variés.

Le travail qui suit est organisé en 3 principaux chapitres :

- Chapitre 1 : est dédié aux fondements théoriques. Il présente une formalisation détaillée du problème de partitionnement de graphes en p classes à peu près équitables 2.1, ainsi que les définitions relatives aux voisinages de solutions et aux structures de données nécessaires à l'implémentation 2.2.
- Chapitre 2 : expose la méthodologie de résolution. Il détaille les différents algorithmes développés (exacts ou approchés) tout en justifiant les choix de conception et les décisions prises 3.
- Chapitre 3 : est consacré à la phase expérimentale. Il décrit le protocole de test, présente les résultats obtenus et des analyses comparatives des performances 4 et 5.

2 Définitions et fondements théoriques

2.1 Formalisation du problème

2.1.1 Instance

Une instance pour notre problème est un graphe non orienté $G = (N, A)$ avec $|N| = n$, $|A| = m$, où N est l'ensemble des sommets numérotés de 1 à n, A est l'ensemble des arêtes et $n, m \in \mathbb{N}$. Le graphe est pondéré avec des poids réels positifs (par défaut 1). Un sommet $u \in N$ est un voisin d'un autre sommet $v \in N$ s'il existe une arête $(u, v) \in A$. Nous notons le nombre de voisins d'un sommet $v \in N$ par $deg(v)$. Le poids d'une arête (u, v) est également noté par w_{uv} avec w une fonction de A à $\mathbb{N}$.

2.1.2 Solution

Une partition $C = \{C_1, \dots C_p\} \subseteq \mathscr{P}(N)$ d'un graphe G en p classes avec $2 \leqslant p \leqslant n$ pour $p \in \mathbb{N}$ est un ensemble d'ensembles de sommets, tel que

$$1. \quad \bigcup_{k=1}^{p} C_k = N,$$
$$2. \quad C_i \cap C_j = \varnothing \quad \forall\, i \neq j.$$

On note $C(u) = k$, si un sommet $u \in N$ est dans la classe k pour une configuation C. Tous les sommets doivent être dans exactement une classe. Sans la contrainte d'équilibre, le nombre total de solutions réalisables est p^n. Il est intéressant d'étudier plusieurs valeurs de p, car le nombre de classes change complètement la structure du problème. Il y a des applications où deux classes suffisent, par exemple le calcul parallèle sur deux processeurs. Cependant, dans la plupart des applications réelles, deux classes ne suffisent presque jamais. Par exemple, dans un circuit électronique, on doit souvent diviser le graphe en plusieurs zones, ce qui correspond à des valeurs de p plus grandes.

2.1.3 La fonction objectif

Le but du problème est de trouver la partition C de telle sorte que la somme des poids des arêtes entre sommets n'appartenant pas à la même classe soit minimale. Le poid d'un arête représent un coût dans les applications réeles. Plus il y a d'arêtes qui traversent les classes, plus il faut de communications, de transferts de données ou de connexions longues. Réduire ces arêtes permet donc d'obtenir un partition plus efficace. On note $A_{ij} := \{\{u, v\} \in A : u \in C_i, v \in C_j, i \neq j\}$ l'ensemble des arêtes inter-classes, et $A_{ij}^k := \{\{u, v\} \in A : u, v \in C_i\}$ l'ensemble des arêtes intra-classes de la classe k.
La fonction objectif est

$$f(C) = \sum_{\{u,v\} \in A_{ij}} w_{uv}.$$

2.1.4 La contrainte d'équité

Il faut que les sommets du graphe soient répartis à peu près équitablement dans les p classes. Cela empêche les solutions triviales (une classe pour tous les noeuds) qui ne sont pas utiles dans la pratique et permet la comparabilité des solutions. Dans beaucoup de domaines, on veut que chaque classe ait à peu près la même taille, pour répartir la charge de travail ou les composants de manière équilibrée. Si on exige un équilibre parfait (toutes les classes ont exactement la même taille), le problème est algorithmiquement beaucoup plus difficile. C'est pourquoi on permet de petites variations. C'est aussi obligatoire pour permettre de faire des mouvements métaheuristiques et ne pas les bloquer. On définit $L_{max} := \lceil n/p * (1 + \epsilon) \rceil$ avec $\epsilon \geqslant 0$.

Pour une solution, C on ajoute la contrainte suivante :

$$|C_k| \leqslant L_{max} \quad \forall k = 1, \dots p. \tag{1}$$

Si $\epsilon = 0$, alors on a une partition parfaitement équilibrée ($|N|/p$ sommets dans une classe). Plus ϵ est grand, plus le problème se simplifie. Le nombre de solutions réalisables change avec la restriction. Par exemple, si on choisit $\epsilon = 0$ petit, tel que

$$||C_i| - |C_j|| \leqslant 1, \ \forall\ 1 \leqslant i, j \leqslant p, i \neq j.$$

On a pour $p = 2$:

$$\text{nombre de solutions réalisables} = \begin{cases} \binom{n}{n/2}, & \text{si } n \text{ pair ,} \\ 2\binom{n}{(n+1)/2}, & \text{si } n \text{ impair.} \end{cases}$$

La contrainte limite fortement les métaheuristiques, alors [AR04] propose d'utiliser une autre formulation pour une contrainte plus douce mais en imposant une pénalité si une classe n'a pas la cardinalité imposée. On calcule :

$$pen = \mu \sum_{k=1,\ldots,p} ||C_k| - n/p| \ \text{ avec } \mu > 0.$$

Une petite valeur pour μ impose un équilibre moins important tandis qu'une grande impose un équilibre très important. La fonction objectif devient de minimiser $f + pen$. Le nombre de solutions réalisables reste donc le même p^n, car la contrainte douce n'interdit aucune solution. Elle évalue uniquement la qualité où le caractère négatif d'une solution. Pour notre projet, nous choisissons la version avec la contrainte (1), car elle garantit directement que chaque classe reste dans la taille autorisée.

2.1.5 Exemple

Considérons le graphe non orienté suivant avec $n = 6$ sommets et $m = 7$ arêtes.

$$A = \{(1,2,3),\ (1,3,1),\ (2,3,2),\ (2,4,4),\ (3,5,5),\ (4,6,2),\ (5,6,3)\}$$

où chaque triplet (u, v, w) représente une arête entre u et v de poids w. Nous cherchons une partition en $p = 3$ classes, avec une contrainte d'équilibre.

$$|C_k| \leqslant \lceil n/p * (1+\epsilon) \rceil = \lceil 2(1+\epsilon) \rceil .$$

Pour $\epsilon = 0$, on obtient $L_{\max} = 2$, donc chaque classe peut contenir au plus 2 sommets.

Une solution réalisable est par exemple :

$$C_1 = \{1,4\}, \qquad C_2 = \{2,5\}, \qquad C_3 = \{3,6\}.$$

Les arêtes inter-classes sont

$$(1,2,3),\ (1,3,1),\ (2,3,2),\ (2,4,4),\ (3,5,5),\ (4,6,2),\ (5,6,3)$$

Toutes les arêtes relient des sommets de classes différentes, donc

$$f(C) = 3 + 1 + 2 + 4 + 5 + 2 + 3 = 20.$$

Certaines solutions sont différentes dans un sens mathématique, mais les mêmes en réalité. Par exemple

$$C' = \{\{5,2\},\ \{3,6\},\ \{1,4\}\}.$$

Une solution irréalisable est par exemple

$$C_1 = \{1,2,3\}, \quad C_2 = \{4,5\}, \quad C_3 = \{6\}$$

Ici, $|C_1| = 3 > L_{\max} = 2$, donc la solution ne satisfait pas la contrainte d'équilibre.

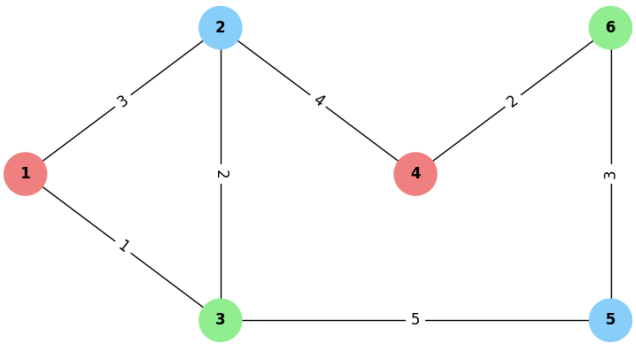


FIGURE 1 – Graphe d'exemple avec les classes colorées

2.1.6 Des voisinages et un mouvement élémentaire

Le voisinage d'une solution C est l'ensemble

$$V(C) = \{C' | C' \text{ une solution qui est obtenue à partir de C par un seul mouvement élementaire}\}.$$

Dans ce qui suit, nous définissons plusieurs mouvements élémentaires dont nous aurons besoin pour les différents algorithmes. Avec les mouvements définis ci - dessus, toute solution est accessible depuis n'importe quelle autre.

PICK'N'DROP Un PICK'N'DROP consiste à effectuer un déplacement d'un seul sommet u d'une classe C_i vers une classe C_j avec $i \neq j$. Ce mouvement produit une nouvelle configuration C', telle que :

$$C'(u) = j \text{ et } C'(v) = C(v) \ \forall v \neq u.$$

C et C' ne diffèrent que par la position de u, selon qu'il appartient ou non à la classe i. Autrement dit, toutes les solutions qui se distinguent par un seul sommet. Le voisinage d'une solution C est de cardinalité $|V(C)| = n * (p-1)$ car un sommet u avec $C(u) = i$ peut être déplacé dans toutes les autres classes $k \neq i$ ($p-1$ possibilités) et il y a n sommets. Il faut faire attention avec ce mouvement car on peut obtenir une solution irréalisable.

SWAP Un SWAP contient un échange des classes des deux sommets distincts u v, appartenant initialement à deux classes différentes C_i et C_j avec $i \neq j$. Ce mouvement produit une nouvelle configuration C', telle que :

$$C'(u) = C(v), \qquad C'(v) = C(u) \qquad \text{et} \qquad C'(w) = C(w) \quad \forall w \notin \{u, v\}$$

C et C' diffèrent exactement par l'échange des classes de deux sommets. Avec ce mouvement, toute solution obtenue par une autre respecte la contrainte d'équilibre donc reste réalisable. La taille du voisinage est :

$$|V(C)| = |\{(u, v) \in V^2 : u < v, \ C(u) \neq C(v)\}| = \sum_{1 \leqslant i < j \leqslant p} |C_i| \cdot |C_j|.$$

(Chaque sommet de C_i peut être échangé avec n'importe quel autre sommet de C_j pour tout $j \neq i$. On ne considère que les cas où $i < j$ afin d'éviter de compter deux fois les mêmes échanges.)

3-SWEEP Un mouvement 3-SWEEP pour $p > 2$ consiste à faire un cycle entre trois sommets distincts u, v et w, appartenant à trois classes différentes C_i, C_j et C_k avec $i \neq j \neq k$. Le principe est de déplacer chaque sommet vers la classe du suivant dans le cycle :

$$C'(u) = C(v), \qquad C'(v) = C(w), \qquad C'(w) = C(u),$$

et pour tout autre sommet x, on garde $C'(x) = C(x)$. Ce mouvement modifie exactement trois sommets, mais il garde la taille des classes inchangée : chaque classe perd un sommet et en gagne un autre. Ainsi, la contrainte d'équilibre est toujours respectée. La taille du voisinage associé est :

$$|V(C)| = \sum_{1 \leqslant i < j < k \leqslant p} |C_i| \cdot |C_j| \cdot |C_k|.$$

Ce mouvement permet d'explorer des solutions plus éloignées qu'un simple PICK'N'DROP ou un SWAP, tout en restant dans l'ensemble des solutions réalisables.

2-OPT Le mouvement 2-OPT est une extension du SWAP. Il consiste à choisir deux sommets u et v dans une classe C_i et deux sommets x et y dans une classe C_j avec $i \neq j$. On échange ensuite les classes de ces quatre sommets :

$$C'(u) = j, \quad C'(v) = j, \quad C'(x) = i, \quad C'(y) = i.$$

Ce mouvement modifie donc quatre sommets en même temps. Comme chaque classe perd deux sommets et en gagne deux, la contrainte d'équilibre reste toujours satisfaite.

La taille du voisinage est :

$$|V(C)| = \sum_{1 \leqslant i < j \leqslant p} \frac{|C_i|(|C_i| - 1)}{2} \cdot \frac{|C_j|(|C_j| - 1)}{2}.$$

Le 2-OPT permet de faire des changements plus importants dans la partition, ce qui peut aider à sortir de minima locaux dans les méthodes de recherche locale.

2.2 Structures de données et outils de base

Pour notre expérimentation, nous avons implémenté les algorithmes. Dans la suite, nous présenterons les structures différentes utilisées.

2.2.1 Représentation du graphe

[AMO93] (S.31ff) propose des représentations différentes pour un graphe pondéré. Dans la suite, nous discuterons deux façons différentes :

Liste d'adjacence Une liste d'adjacence représente un graphe G en associant à chaque sommet $v \in N$ la liste de ses voisins. Pour chaque arête $(u, v) \in E$, on stocke le couple (v, w_{uv}) dans la liste de u.

Matrice d'adjacence Une matrice d'adjacence représente un graphe G par une matrice $A \in \mathbb{R}^{n \times n}$ telle que

$$A_{uv} = \begin{cases} w_{uv}, & \text{si } (u, v) \in A, \\ 0, & \text{sinon.} \end{cases}$$

Table 1 – Comparaison des représentations d'un graphe pondéré

Critère	**Liste d'adjacence**	**Matrice d'adjacence**
Mémoire	$O(n + m)$	$O(n^2)$
Accès aux voisins de v	$O(\deg(v))$	$O(n)$
Existence d'une arête (u, v)	$O(\deg(u))$	$O(1)$
Boucle sur toutes les arêtes	$O(m)$	$O(n^2)$
Type de graphe adapté :	Graphe creux (m petit)	Graphe dense (m grand)

Reprenons l'exemple (2.1.5), on a pour la liste d'adjacence

$$\begin{aligned}
&1 : (2,3),(3,1)\\
&2 : (1,3),(3,2),(4,4)\\
&3 : (1,1),(2,2),(5,5)\\
&4 : (2,4),(6,2)\\
&5 : (3,5),(6,3)\\
&6 : (4,2),(5,3)
\end{aligned}$$

et pour la matrice d'adjacence

$$A_{\text{haut}} = \begin{pmatrix} - & 3 & 1 & 0 & 0 & 0 \\ & - & 2 & 4 & 0 & 0 \\ & & - & 0 & 5 & 0 \\ & & & - & 0 & 2 \\ & & & & - & 3 \\ & & & & & - \end{pmatrix}$$

Dans un graphe non orienté, la matrice d'adjacence est symétrique. On ne peut donc stocker que la matrice triangulaire supérieure ou inférieure. Cela permet de diviser par deux les besoins en mémoire $O((n(n-1))/2))$. Nous avons testé notre instance sur la densité des graphes. La densité est la fraction

$$\frac{\text{nb d'arêtes}}{\text{nb max d'arêtes}}.$$

— petites instances ($n \leqslant 30$) : 0.7 à 0.99
— taille moyenne instances ($50 \leqslant n \leqslant 100$) : 0.4
— grandes instances ($n \geqslant 500$) : 0.02 à 0.2

Nous conclurons que la matrice d'adjacence serait mieux pour les petites instances, mais très coûteuse pour les plus grandes. Nous choisissons finalement d'opter pour une liste d'adjacence afin de représenter nos graphes puisque nous serons ramener à parcourir également les voisins d'un sommet avec certains algorithmes.

2.2.2 Représentation d'une partition

Il existe plusieurs façons de coder une solution pour ce problème. Nous comparons ici les deux approches les plus répandues.

TABLE 2 – Comparaison entre tableau des classes et représentation par ensembles

Opération	**Tableau des classes**	**Ensembles de sommet**
Représentation	Un tableau de taille n où chaque entrée indique la classe à laquelle appartient un sommet : classe$[v] = k \in \{0, \ldots, p-1\}$	$\{C_1, C_2, \ldots, C_p\}$ avec $C_i \subseteq V$
Implémentation	`array('b', [-1]*n)`	p ensembles : `set()`
Exemple	`classe` $= [1,2,3,1,2,3]$	$C = \{\{1,4\},\ \{2,5\},\ \{3,6\}\}$
Mémoire	$O(n)$	$O(n)$
Connaître la classe d'un sommet v	$O(1)$	$O(p)$ (chercher v dans les ensembles)
Changer la classe d'un sommet v	$O(1)$	$O(1)$

Nous avons choisi le tableau des classes pour assurer un accès rapide à la classe d'un sommet, une mise à jour rapide pendant un algorithme et un calcul rapide du coût du cut.

Ainsi, une solution est entièrement décrite par le vecteur :

$$C = (C[0], C[1], \ldots, C[n-1]).$$

Espace de recherche : L'ensemble des solutions codables est : $ER = \{0, 1, \ldots, p-1\}^n$ avec $|ER| = p^n$ comme indiqué avant.
Cet espace contient toutes les solutions réalisables, mais également des solutions non réalisables. On a donc,

$$\mathcal{S}_{\text{réalisables}} \subseteq ER,$$

mais

$$ER \neq \mathcal{S}_{\text{réalisables}}.$$

Toute solution réalisable est accessible depuis n'importe quelle configuration initiale en modifiant les classes sommet par sommet.

2.2.3 Les calculs incrémentaux

La valeur du cut d'une nouvelle configuration à partir d'une configuration et d'un mouvement élémentaire On veut que les algorithmes soient rapides et utilisables. Il faut déterminer la valeur d'une nouvelle configuration à partir d'une configuration et d'un mouvement élémentaire peu coûteux. Le calcul de la valeur objectif d'une solution C coût $O(n+m)$ avec nos représentations. Avec un mouvement élémentaire, seule une petite partie du graphe change.

PICK'N'DROP Le mouvement PICK'N'DROP consiste à déplacer un sommet u d'une classe i vers une classe j avec $i \neq j$. Lors d'un mouvement PICK'N'DROP, seules les arêtes incidentes à u peuvent modifier la valeur du cut. Pour chaque voisin $v \in N(u)$, la variation de la fonction objectif est :

$$\Delta = \sum_{v \in N(u)} \begin{cases} +w_{uv}, & \text{si } C(v) = i \quad (\text{intra} \to \text{inter}), \\ -w_{uv}, & \text{si } C(v) = j \quad (\text{inter} \to \text{intra}), \\ 0, & \text{sinon.} \end{cases}$$

La nouvelle valeur de la fonction objectif est : $f(C') = f(C) + \Delta$. La complexité est alors $O(\deg(u))$.

SWAP Le mouvement SWAP échange les classes de deux sommets u v. Comme pour PICK'N'DROP, seules les arêtes incidentes à u et v peuvent modifier la valeur du cut. La variation totale est :

$$\Delta = \Delta(u) + \Delta(v).$$

La nouvelle valeur de la fonction objectif est $f(C') = f(C) + \Delta$. La complexité du mouvement est $O(\deg(u) + \deg(v))$.

3-SWEEP Un mouvement 3-SWEEP change seulement la classe de trois sommets u, v et w. La nouvelle valeur est alors :

$$f(C') = f(C) + \Delta(u) + \Delta(v) + \Delta(w).$$

La complexité est en $O(\deg(u) + \deg(v) + \deg(w))$.

2-OPT Le mouvement 2-OPT modifie les classes de quatre sommets u, v, x et y. La nouvelle valeur est alors :

$$f(C') = f(C) + \Delta(u) + \Delta(v) + \Delta(x) + \Delta(y).$$

La complexité est en $O(\deg(u) + \deg(v) + \deg(x) + \deg(y))$.

3 Méthodes de résolution

3.1 Méthodes exactes

3.1.1 Énumération explicite

L'algorithme d'énumération explicite explore toutes les partitions possibles d'un graphe à n sommets en p classes, teste si la solution est réalisable avec les contraintes du problème et calcule la valeur du cut. Il conserve la meilleure solution rencontrée. Cet algorithme permet de trouver la solution optimale d'une instance, mais il a une complexité de $O(p^n(n+m))$ (calculer la valeur du cut $= O(n+m)$). L'algorithme d'énumération n'est applicable que sur les petites instances (au plus 30 sommet). On peut utiliser une borne initiale d'une solution réalisable pour réduire le temps de calcul.

Algorithme 1 : Algorithme d'énumération explicite des partitions en p classes avec borne initiale

Entrée : Graphe G à n sommets, nombre de classes p, ϵ, partition initiale C^{init}
Sortie : C^* : meilleure partition trouvée

```
Fonction Enumeration(G, p, ε)
Initialiser best_val ← f(C^init)
Initialiser best_assignment ← C^init
foreach affectation (c_0, ..., c_{n-1}) ∈ {0, ..., p-1}^n do
    Construire une solution C en posant C(v) ← c_v pour tout v
    if C n'est pas réalisable then
        continuer
    end
    Calculer val ← f(C)
    if val < best_val then
        best_val ← val
        best_assignment ← (c_0, ..., c_{n-1})
    end
end
Construire C* à partir de best_assignment
Calculer f(C*)
return C*
```

3.2 Méthodes heuristiques locales

3.2.1 Descente de gradient (algorithme glouton du gradient sans optimisation)

L'algorithme décrit dans 2 est la version récursive de l'algorithme du gradient, telle qu'introduite par M. Corsini dans [Cor26]. La méthode gloutonne de recherche locale consiste à explorer le voisinage d'une solution initiale afin d'identifier potentiellement une autre qui est meilleure, qui améliore la valeur de la fonction objectif. Comme c'est une méthode gloutonne, elle ne garantit pas l'optimalité globale.

Algorithme 2 : Gradient (version récursive)

Entrée : Solution S, voisinage V
Sortie : S^* : meilleure solution (partition) trouvée

```
Fonction Gradient(G, V)
Initialiser S* ← S
Initialiser f* ← f(S)
foreach X ∈ V(S) do
    if f(X) < f* then
        S* ← X
        f* ← f(X)
    end
end
if S* = S then
    return S*
else
    return Gradient(S*, V)
end
```

3.2.2 Algorithme de Kernighan & Lin (KL)

L'algorithme KL est une heuristique de recherche locale pour le partitionnement ($p = 2$) équilibré des graphes.

Pour une partition réalisable initiale de $p = 2$, soient A et B deux sous-ensembles de V :

— Si la cardinalité de V est paire, V est divisé en deux sous-ensembles de taille égale.

— Sinon, la différence de taille entre les deux sous-ensembles est au plus égale à 1.

.

Après, il sélectionne de manière itérative des paires de sommet (i, j) appartenant à différentes classes, $i \in A$ et $j \in B$. Ensuite, les permute afin de maximiser le gain de la fonction objectif (valeur de coupe).

Pour chaque sommet v, on définit la mesure :

$$D(v) = E(v) - I(v),$$

où $E(v)$ est le poids des arêtes incidentes à v qui contribuent au poids inter-classes, et $I(v)$ est le poids des arêtes incidentes à x qui contribuent au poids intra-classe.

Le gain associé à l'échange des sommets i et j est :

$$g(i, j) = D(i) + D(j) - 2w_{ij},$$

où w_{ij} est le poids l'arête (i, j), si l'arête n'existe pas $w_{ij} = 0$.

A chaque passage, une séquence de swaps est construite et les gains cumulés sont calculés. Seul le préfixe maximisant le gain total

$$G_k = \sum_{t=1}^{k} g(i_t, j_t)$$

est conservé. Le processus s'arrête lorsque le gain cumulatif maximal est non positif.

A noté que puisque KL est une heuristique, l'optimalité globale n'est pas garantie.

En ce qui concerne la complexité, la version théorique de l'algorithme de KL fonctionne dans $\mathcal{O}(n^2 log(n))$ par passage, [Wik26b]. Le facteur logarithmique provient de l'utilisation de structures de données efficaces (telles que les files d'attente prioritaires) pour sélectionner la paire à gain maximal à chaque étape.

En pratique, deux versions KL ont été considérées avec des structures de données différents pour choisir la meilleure paire :

— Énumération par brute force : toutes les paires déverrouillées sont examinées à l'aide de boucles imbriquées sur les deux classes. Cela entraîne une complexité cubique dans le pire des cas, $\mathcal{O}(n^3)$ par passage.

— Version de la file d'attente prioritaire ('heap') : un max-heap stocke les gains candidats, permettant la récupération du meilleur swap dans $\mathcal{O}(log(n))$ au lieu de $\mathcal{O}(n^2)$ par itération. En poussant des gains négatifs ($-g$) dans le heapq de Python (qui par défaut est un min-heap), le gain maximum est maintenu à la racine. Les gains absolus peuvent ensuite être récupérés selon les besoins. Cette structure évite de recalculer tous les gains à chaque étape.

Algorithme 3 : Algorithme de Kernighan & Lin (voisinage SWAP)

Entrée : Graphe $G = (V, E)$, partition initiale $S_0 = \langle C_1, C_2 \rangle$
Sortie : S^* : partition améliorée

```
Fonction KernighanLin(G, S_0)
foreach x ∈ V do
    Calculer D_x = E_x − I_x
end
gain ← 1
while gain > 0 do
    L_a ← [], L_b ← [], L_g ← []
    for k = 1 to min(|C_1|, |C_2|) do
        Choisir a ∈ C_1, b ∈ C_2 maximisant δ_{a,b} = D_a + D_b − 2w_{a,b}
        Ajouter a à L_a, b à L_b, δ_{a,b} à L_g
        Marquer a et b comme verrouillés
        foreach x ∈ V\{a, b} non verrouillé do
            Mettre à jour D_x
        end
    end
    Calculer les sommes cumulées G_i = Σ_{j=1}^{i} L_g[j]
    gain ← max_i G_i
    Soit k* = arg max_i G_i
    if gain > 0 then
        C_1 ← (C_1\L_a[1 : k*]) ∪ L_b[1 : k*]
        C_2 ← (C_2\L_b[1 : k*]) ∪ L_a[1 : k*]
    end
end
return ⟨C_1, C_2⟩
```

3.2.3 Gradient avec heuristique

Afin d'accroître l'efficacité de l'algorithme de descente de gradient présenté dans 3.2.1, nous introduisons un paramètre de recherche locale noté k [Cor26]. Le principe est simple, nous proposons de parcourir les solutions voisines et de s'arreter à la $k$ème amélioration trouvée plutot que d'explorer tout le voisinnage pour identifier un optimum local. Tout en espérant de réduire la complexité de $\mathrm{O}(S \times I \times m \times p \times (n + m))$ (associé à la descente de gradient recursive) à $\mathrm{O}(S \times I \times |V_v| \times (n + m))$ avec S le nombre de fois d'executions de l'algorithme, I nombre d'itérations représentant respectivement le nombre de fois que "gradient" s'appelle récursivement et le nombre de fois qu'on met à jour la solution courrante, n le nombre de sommets, p le nombre de partitions, m le nombre d'aretes du graphe et $|V_v|$ le nombre de voisins visités (tel que $k \leqslant |V_v| \leqslant m \times p$).

Algorithme 4 : Gradient Heuristique (arrêt à la k-ième amélioration)

Entrée : Configuration initiale C_0, entier $k > 0$, voisinage V
Sortie : C_i : configuration finale (optimum local après k améliorations)

```
i ← 0
fini ← faux
while ¬fini do
    Calculer f(C_i)
    j ← 0
    sauvegarde ← Nul
    // Exploration partielle du voisinage
    foreach C ∈ V(C_i) et tant que j < k do
        if f(C) < f(C_i) then
            j ← j + 1
            sauvegarde ← C
        end
    end
    if sauvegarde ≠ Nul then
        C_{i+1} ← sauvegarde
        i ← i + 1
    else
        fini ← vrai
    end
end
return C_i
```

Nous avons priviligié cette heuristique parce qu'elle nous évite d'une part l'exploration exhaustive du voisinnage, d'autre part, tout en évitant le biais potentiels liés à un découpage arbitraire en blocs. Une segmentation mal définie pourrait louper les meilleures trajectoires d'amélioration. Toutefois, nous restons vigilants par rapport au choix du paramètr k puisqu'une faible valeur de ce dernier entrainerait éventuellement une convergence prématurée vers un optimum local médiocre.

3.3 Méta-heuristiques

3.3.1 Le recuit simulé

Le recuit simulé est une métaheuristique stochastique inspirée du refroidissement d'un métal chauffé. Une solution du problème correspond à un état du système, et la fonction objectif joue le rôle de l'énergie. Quand la température est haute, l'algorithme accepte aussi des solutions moins bonnes. Cela permet d'éviter de rester bloqué dans un minimum local. Quand la température baisse, l'algorithme devient plus strict et accepte de moins en moins de mauvaises solutions. À la fin, il se stabilise sur une bonne solution.

L'algorithme commence avec une solution initiale X et une température initiale T_0. Pour chaque niveau de température, on génère plusieurs solutions voisines. Une nouvelle solution X' est toujours acceptée si elle améliore la fonction objectif. Si elle est moins bonne, elle peut être acceptée avec une probabilité donnée par la règle de Metropolis [Met+53] :

$$p = e^{-\Delta f/T}.$$

Dans cette formule, f représente la fonction objectif du problème. Elle mesure la qualité d'une solution : plus f est petite, meilleure est la solution. La quantité $\Delta f = f(X') - f(X)$ indique donc si la nouvelle solution X' est meilleure ou moins bonne que la solution actuelle X.

La température T est un paramètre qui contrôle le comportement de l'algorithme. Quand T est grande, l'algorithme accepte plus facilement des solutions moins bonnes. Quand T devient petite, il devient plus strict et n'accepte presque plus de détériorations. Enfin, $p \in [0, 1]$ est la probabilité d'accepter une solution moins bonne pour sortir d'un minimum local. Si Δf est grand ou si la température

est basse, cette probabilité devient très faible. Cette règle permet à l'algorithme d'explorer au début, puis de se stabiliser progressivement vers une bonne solution.

Après chaque phase, la température est diminuée avec une fonction strictement décroissante $g(T)$ à valeur positive qui va faire tendre la température T vers 0, souvent de manière géométrique :

$$T_{t+1} = \alpha T_t, \quad \text{avec } 0 < \alpha < 1.$$

Une valeur proche de 1 correspond à un refroidissement lent, ce qui permet une exploration plus longue de l'espace des solutions. Une valeur plus petite accélère le refroidissement et rend l'algorithme plus rapide mais moins exploratoire. Le processus s'arrête quand la température devient très basse ou quand il n'y a plus d'amélioration. L'algorithme renvoie alors la meilleure solution trouvée pendant la recherche.

Pour choisir la solution initiale, nous avons décidé de prendre une configuration aléatoire et de faire confiance au mécanisme du recuit simulé pour améliorer progressivement cette solution. Kirkpatrick [KGV83] proposent une méthode analitique pour choisir la température initiale T_0. L'idée est de fixer à l'avance une probabilité $p \in [0, 1]$ d'accepter une solution plus mauvaise au début de l'algorithme. Pour cela, on génère un échantillon aléatoire d'environ 10% des mouvements possibles de type PICK'N'DROP et on calcule la valeur moyenne des détériorations admissibles.

$$\Delta f_{\text{moy}} = \frac{1}{|S|} \sum_{X' \in S} \left| \Delta f(X, X') \right|,$$

où S est l'ensemble des voisins échantillonnés réalisables. La température initiale est alors choisie de manière à vérifier :

$$p = e^{-\Delta f_{\text{moy}}/T_0}.$$

En résolvant cette équation, on obtient :

$$T_0 = -\frac{\Delta f_{\text{moy}}}{\ln(p)}.$$

Cette méthode permet de calibrer automatiquement T_0 en fonction du problème et garantit que l'algorithme explore suffisamment l'espace des solutions au début. Dans nos expériences, nous avons choisi $p = 0.8$ qui permet d'obtenir un bon compromis entre exploration et stabilité. La première condition a pour but d'itérer le processus jusqu'à l'obtention d'une solution satisfaisante. Plusieurs choix sont possibles, mais on choisit d'arrêter l'algorithme lorsque la température devient suffisamment faible. On fixe alors une température finale $T_{\text{final}} = T_0 * 10^{-2}$ et l'algorithme s'arrête dès que $T \leqslant T_{\text{final}}$. Ce critère est suffisamment car lorsque la température est très basse, la probabilité d'accepter une solution plus mauvaise devient presque nulle, et l'algorithme se comporte comme une recherche locale classique. La deuxième condition représente la recherche à température constante. Elle doit être suffisamment longue pour permettre à l'algorithme d'explorer correctement le voisinage et d'appliquer la règle de Metropolis. En pratique, on fixe souvent un nombre d'itérations N_{iter} pour chaque niveau de température. On choisit simplement N_{iter} comme un polynôme de n où n est le nombre de sommets du graphe. À chaque itération, on explore alors arbitrairement un voisin de type PICK'N'DROP.

Algorithme 5 : Recuit simulé

Entrée : Solution initiale X, température initiale T_0, fonction de refroidissement g, voisinage V, nombre de classes p, ϵ

Sortie : X_{best} : meilleure solution trouvée

```
Fonction RecuitSimule(X, T_0, p, ε)
Initialiser X_best ← X
Initialiser f_min ← f(X)
Initialiser T ← T_0
while cond1 do
    while cond2 do
        Choisir X_{t+1} dans V(X)
        Calculer Δf = f(X_{t+1}) − f(X)
        if Δf < 0 then
            X ← X_{t+1}
            if f(X_{t+1}) < f_min then
                X_best ← X_{t+1}
                f_min ← f(X_{t+1})
            end
        end
        else
            Tirer p uniformément dans [0, 1]
            if p ⩽ e^{−Δf/T} then
                X ← X_{t+1}
            end
        end
        t = t+1
    end
    T ← g(T)
end
return X_best
```

3.3.2 Algorithme génétique simple (AGS)

L'algorithme génétique simple fonctionne sur une population de solutions candidates (appelées individus ou chromosomes) et évolue au fil des générations pour maximiser une fonction de fitness en utilisant la sélection, le croisement et la mutation. [Wik26a]

Représentation Chaque solution du problème de partitionnement est codée sous la forme d'un chromosome C appartenant à l'espace des solutions réalisables. Dans le cadre de ce problème, une solution correspond à une affectation de chaque sommet du graphe à l'une des p classes.

Ainsi, un individu est représenté par un vecteur de taille n, où chaque composante indique la classe associée au sommet correspondant. Formellement, un chromosome est défini par :

$$C = (C[0], C[1], \ldots, C[n-1]), \quad \textit{avec } C[i] \in \{0, \ldots, p-1\}.$$

Fitness fonction L'objectif est de résoudre un problème de minimisation du cut. Comme l'algorithme génétique repose sur une maximisation, on transforme la fonction objectif en fitness.

Pour une solution C, le cut est défini comme la somme des poids des arêtes inter-classes. On cherche à le minimiser tout en respectant la contrainte d'équilibre contrôlée par ϵ.

La fitness est définie par :

$$f(C) = C_{\max} - \text{cut}(C),$$

où $C_{\max}$ est choisi tel que $f(C) \geqslant 0$.

Seules les solutions réalisables sont évaluées, afin de favoriser les partitions équilibrées avec un cut minimal.

Sélection Les individus sont sélectionnés en fonction de leur fitness. La probabilité $p_i \in [0, 1]$ de sélectionner un individu i est est sa fitness pondérée par la somme des fitness de tous les individus :

$$p_i = \frac{f_i}{\sum_j f_j}$$

Cela favorise de meilleures solutions tout en maintenant la diversité dans la population.

Crossover (Recombinaison) Deux chromosomes parentaux sont combinés pour produire une nouvelle descendance. Pour un chromosome de longueur l, un point de croisement $k \in \{1, \ldots, l-1\}$ est choisi et les parties sont échangées (SWAP) :

$$\text{child}_1 = (x_1^{(1)}, \ldots, x_k^{(1)}, x_{k+1}^{(2)}, \ldots, x_l^{(2)})$$

$$\text{child}_2 = (x_1^{(2)}, \ldots, x_k^{(2)}, x_{k+1}^{(1)}, \ldots, x_l^{(1)})$$

Cela correspond à l'intensification (recherche locale).

Mutation Chaque gène est modifié de manière aléatoire avec une faible probabilité p_m (généralement très faible, p.e. ≈ 0.001). Cela introduit la diversité et évite les optima locaux :

$$x_i \leftarrow \begin{cases} 1 - x_i & \text{avec probabilité } p_m \\ x_i & \text{avec probabilité } 1 - p_m \end{cases}$$

La mutation permet d'explorer de nouvelles zones de l'espace de recherche en modifiant parfois un gène au hasard. Cela évite que l'algorithme reste bloqué et garantit que toutes les régions du domaine peuvent être atteintes.

Parameters L'algorithme dépend de plusieurs paramètres :
— taille de lapopulation,
— nombre de générations,
— probabilité de crossover,
— probabilité de mutation.

Critère d'arrêt L'algorithme s'arrête après un nombre fixe de générations ou lorsqu'aucune amélioration n'est observée.

Évolution et convergence La population évolue au fil des générations, et le fitness moyen augmente jusqu'à converger vers une solution optimale ou quasi-optimale.

La Figure 2 illustre les différentes étapes de ce processus.

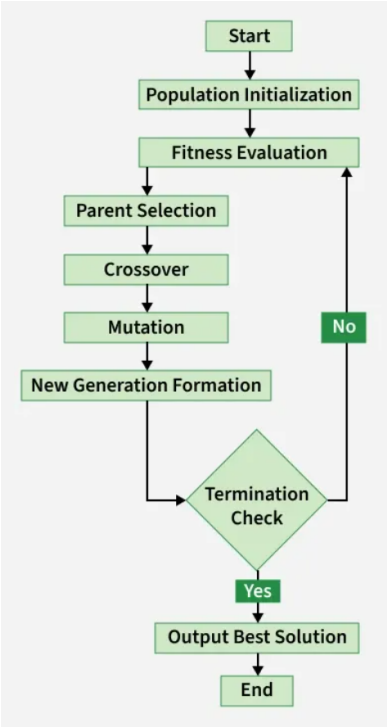


FIGURE 2 – Schéma de fonctionnement d'un algorithme génétique
Source : https://www.geeksforgeeks.org/dsa/genetic-algorithms/

— **Population Initialization :** Générer une population initiale de chromosomes au hasard.
— **Fitness Evaluation :** Évaluer chaque chromosome à l'aide de la fonction de fitness pour mesurer la qualité des solutions.
— **Parent Selection :** Sélectionner les chromosomes parents en fonction de leur fitness à l'aide de méthodes telles que la roulette, le tournoi.
— **Crossover :** Combiner le matériel génétique des parents sélectionnés pour produire une descendance.
— **Mutation :** Appliquer des changements aléatoires aux gènes de la descendance pour maintenir la diversité.
— **New Generation Formation :** Remplacer l'ancienne population par la descendance nouvellement générée.
— **Termination Check :** Arrêter l'algorithme si les critères de terminaison sont satisfaits.
— **Solution de sortie :** Renvoyer le meilleur chromosome obtenu au cours de l'évolution.

Algorithme 6 : Algorithme Génétique Simple (AGS)

Entrée : Taille de population N, taux de mutation p_m, ϵ, fonction objectif $g(x)$
Sortie : x^* : meilleur chromosome

```
Fonction GeneticAlgorithm(N, p_m, ε) ;
Initialiser une population P de N chromosomes aléatoires x ∈ 𝒳 ;
foreach x ∈ P do
    Calculer f(x) = C − g(x) ;
end
while critère d'arrêt non satisfait do
    foreach x_i ∈ P do
        Calculer p_i = f_i / Σ_j f_j
    end
    Sélectionner des parents selon les probabilités p_i
    P_offspring ← []
    for chaque paire de parents (x^(1), x^(2)) do
        Choisir un point de croisement k
        Construire child_1, child_2 par SWAP
        foreach gène i des enfants do
            Muter avec probabilité p_m
        end
        Ajouter les enfants à P_offspring
    end
    foreach x ∈ P_offspring do
        Calculer f(x)
    end
    P ← P_offspring
end
x* ← arg max_{x∈P} f(x) ;
return x*
```

4 Expérimentations et résultats

Pour nos expérimentations, nous avons implémenté les méthodes de résolution avec le language python. Notre implémentation permet de tester toutes les valeurs possibles de p et ϵ, mais, dans le cadre de cette étude, nous nous sommes limités aux cas $2 \leqslant p \leqslant 6$ et $\epsilon \in \{0, \frac{1}{2}, 1\}$ avec un temps limite d'une heure par instance. Enfin, pour l'algorithme de Kernighan–Lin, nous avons seulement testé le cas $p = 2$ et $\epsilon = 0$, car sa structure interne impose une partition strictement équilibrée en deux classes de même taille. Les instances utilisées dans nos tests sont des graphes de tailles variées, allant de 4 jusqu'à 10.000 sommets et elles sont accessibles à l'adresse suivante : https://app.box.com/s/mephvzsznzqbq4sfrdhxec6y3bnr4kx0/folder/632356523. Les codes permettant d'effectuer les expérimentations sont disponibles sur GitHub https://gitlab.emi.u-bordeaux.fr/meta/meta ou sur https://github.com/lafilledepondy/metaHeuristiques (si vous n'etes pas connecté avec votre id du cremi). De plus, la structure de base du code utilisé dans nos tests vient du projet d'optimisation *ROAD* du Master 1, fourni par Thibault Prunet. Nous avons ensuite adapté et complété ce code pour réaliser nos propres méthodes et expériences. Les tests suivants ont été réalisés sur une machine équipée d'une carte graphique NVIDIA RTX 3060 (12 Go), d'un processeur Intel Xeon W-1290 et de 64 Go de mémoire vive, fonctionnant sous un système d'exploitation 64 bits.

Le paramètre ϵ contrôle le niveau d'équilibre entre les classes. Quand ϵ est petit, les classes doivent avoir presque la même taille : il y a donc moins de solutions possibles et les algorithmes sont plus rapides, mais plus limités. Quand ϵ est grand, les classes peuvent être déséquilibrées : le nombre de solutions augmente, ce qui rend la recherche plus longue, mais permet d'obtenir des coupures plus petites.

Dans nos expériences, nous avons choisi les valeurs $\epsilon \in \{0, \frac{1}{2}, 1\}$ pour couvrir trois situations simples : un équilibre strict ($\epsilon = 0$), un équilibre partiel ($\epsilon = \frac{1}{2}$) et un cas très relâché ($\epsilon = 1$).

Pour les petites instances, lorsque $\epsilon = 1$, la contrainte sur la taille des classes devient presque inexistante. Par exemple, pour $n = 25$ et $p = 2$, on obtient :

$$|C_k| \leqslant \lceil 25/2 * (1 + 1) \rceil = 25 \ \forall k \in \{1, 2\}.$$

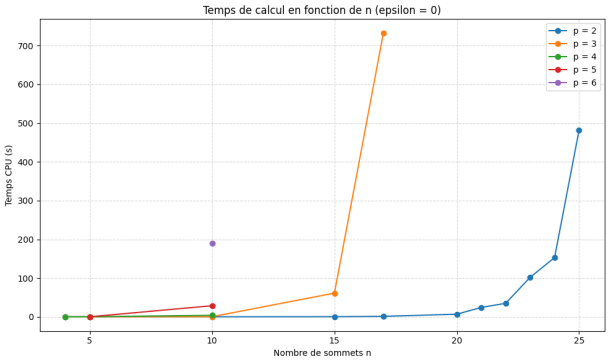
Dans ce cas, on peut choisir la solution triviale qui place tous les sommets dans une seule classe. La valeur de la fonction objectif devient alors 0, puisqu'il n'y a plus aucun arrêt intraclasse.

4.1 Méthodes exactes - Énumération explicite

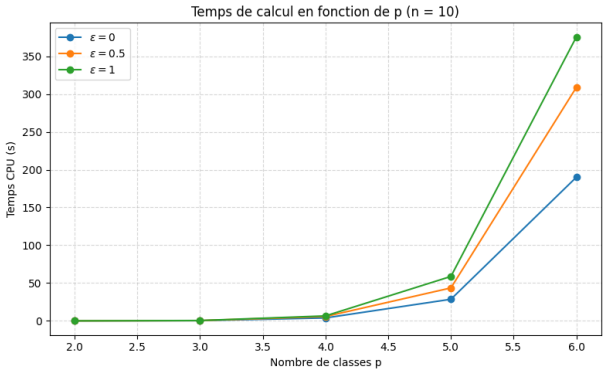
Nous avons testé l'algorithme sur les instances avec $n < 30$. Les résultats numériques sont résumés dans le tableau (15). Déjà pour $p = 3$ et $n \geqslant 20$, l'algorithme n'a pas réussi à trouver une solution optimale dans la limite de temps imposée. On peut observer bien une croissance exponentielle du temps de calcul en fonction du nombre de sommets et du nombre de classes. Cela confirme la complexité théorique en $O(p^n(n+m))$ et limite l'usage aux très petites instances. On peut voir bien que le nombre de classes a une grande influence aussi. Par exemple, pour 10 sommets, passer de $p = 2$ à $p = 6$ multiplie le temps par plus de 50 000. Cette valeur est cohérente avec la croissance théorique :

$$\frac{6^{10}}{2^{10}} = \left(\frac{6}{2}\right)^{10} = 3^{10} = 59\,049.$$

Les résultats suivent l'effet attendu du paramètre ϵ présenté plus haut. Ces résultats justifient pleinement la nécessité d'heuristiques ou de méthodes exactes plus intelligentes.



(a) Temps de calcul en fonction de n (pour $\epsilon = 0$).



(b) Temps de calcul en fonction de p (pour $n = 10$).

FIGURE 3 – Comparaison des temps de calcul selon le nombre de sommets et le nombre de classes.

4.2 Méthodes heuristiques locales

4.2.1 Algorithme du gradient (version récursive)

Etant donné qu'il s'agit d'une heuristique gloutonne dépendante de la solution initiale, nous avons implémenté une stratégie multi-starts dans le but de pallier les risques de convergence prématurée et de stagnation dans les optima locaux.

L'image 4 illustre la vulnérabilité de notre algorithme face aux deux risques évoqués. En effet, si notre solution initiale est située au **"start point 1"**, le processus de descente, s'y trouvera piégé, convergera vers le minimum local et il n'atteindra jamais le minimum global avec le même point de départ. Cette limitation s'explique par la nature de recherche gloutonne ; puisque seuls les déplacements minimisant la fonction objectif qui sont autorisés notre algorithme est incapable de remonter la pente pour ensuite minimiser du coté du minimum global.

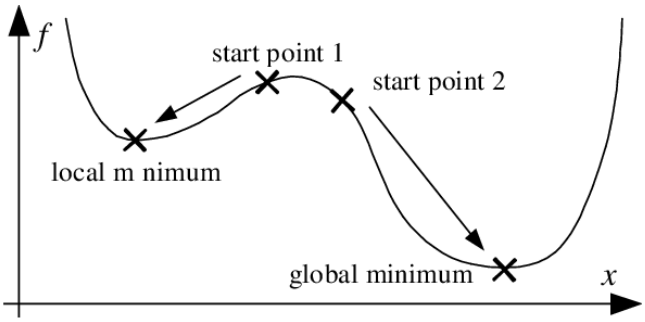


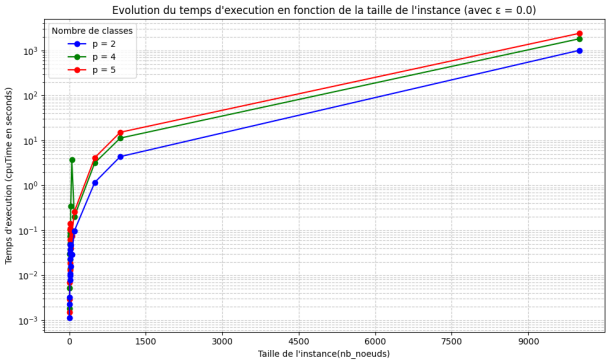
FIGURE 4 – Optimum local vs Optimum global

Afin de contourer cette limitation, la stratégie multi-starts exécute l'algorithme du gradient plusieurs fois en générant à chaque fois une solution initiale aléatoire, dans une fenêtre temporelle limitée à une heure pour chaque instance. Le nombre d'execution par défaut est fixé à 50 fois pour établir un compromis raisonnable entre l'exploration de l'espace de recherche et le temps de calcul. D'après la loi faible des grands nombres [Dat23], avec ce seuil la moyenne emprique se rapproche de la moyenne réelle tout en respectant notre contrainte temporelle.
Concernant le voisinnage nous avons opté pour le pick'n'drop parce qu'il offre plus de liberté de mouvement dans l'espace de recherche, dans l'espoir d'atteindre le minimum global. La flexibilité obtenue en déplaçant un seul sommet d'une classe à une autre nous a permis d'explorer des solutions où les partitions ne sont pas strictement de la même taille augmentant ainsi les chances d'atteindre notre objectif.

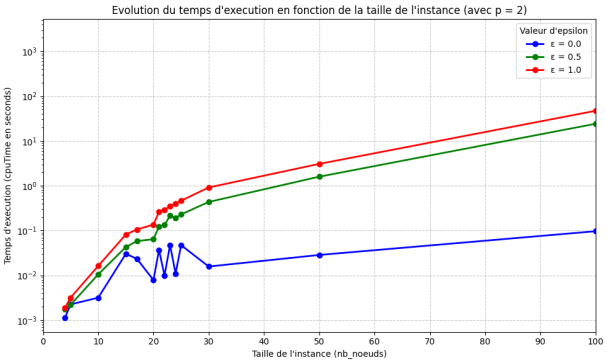
Notre analyse ne se limite pas à une simple recherche de la meilleure valeur retournée **"best"**, mais elle intègre des métriques de dispersion et de fréquence telles que le nombre d'apparition de la meilleure solution, la pire valeur trouvée, la moyenne, la médiane et l'écart type des résultats obtenus, notés **"nb_f", "worst", "mean", "med", "sd"** respectivement. Nous avons noté également le temps d'exécution nécessaire pour chaque instance, noté **"CPU(s)"**.

Les résultats numériques détaillés sont résumés dans la section A dans le tableau 8. Nous remarquons d'une part, que la contrainte d'équitabilité des classes affecte négativement la convergence de l'algorithme, c'est à dire plus nous sommes strictes dans la contrainte d'équitabilité en posant $\epsilon = 0$ plus le nombre d'occurrences de la meilleure solution trouvée est relativement faible par rapport au nombre d'exécutions total. A l'inverse quand $\epsilon = 1$ l'algorithme converge toujours vers la même meilleure solution. Ce constat renforce notre stratégie multi-starts, de 50 exécutions pour notre algorithme du gradient. En effet, surtout dans le cas de la contrainte la plus stricte $\epsilon = 0$, une unique exécution présenterait une probabilité élevée de stagner dans une solution moins bonne et de passer à coté de la meilleure solution trouvée.
Comme expliqué dans l'introduction de cette section, la valeur de ϵ influence directement la difficulté de la recherche et la qualité des solutions. Les résultats obtenus avec l'algorithme du gradient confirment cet effet.

Les graphiques représentés sur 5 illustrent respectivement l'évolution des temps d'exécution en fonction de la taille des instances pour des classes différentes 5a et pour des valeurs ϵ différentes 5b.



(a) Evolution du temps de calcul en fonction de la taille des instances pour différentes classes (avec $\epsilon = 0$).



(b) Evolution du temps de calcul en fonction de la taille des instances pour différentes valeurs d'ϵ (avec $p = 2$)

FIGURE 5 – Comparaison des temps de calcul selon la taille des instances.

Les graphiques illustrent une croissance polynomiale du temps de calcul en fonction de la taille de l'instance.

— Le graphique 5a montre que, pour $\epsilon = 0.0$, le temps d'execution augmente en fonction de la taille des instances et que cette augmentation est amplifiée lorsque le nombre de classes augmente également. En effet en décidant, par exemple, de partitionner les sommets d'un graphe en 5 classes necessite plus de temps que de les répartir en seulement 2 classes. Celà pourra s'expliquer par l'utilisation du pick'n'drop : dans le 1er cas, un sommet du graphe a plus de choix possibles pour sa classe, alors qu ' avec seulement 2 classes, le nombre de possibilités se réduit, ce qui accélère le calcul.
— Le graphique 5b, met en évidence l'impact de la restriction de la contrainte d'équitabilité des classes sur le temps d'exécution des instances (de différentes tailles). Ce qui est tout de suite remarquable, et qui vient confirmer les résultats numériques interprétés précédemment, que plus on relache la contrainte d'équitabilité plus on enregistre une augmentation des temps de calcul .

4.2.2 Kernighan & Lin (KL)

Nous avons appliqué l'algorithme de KL (les deux versions) à nos instances de graphe et comparé notre implémentation avec la version fournie par la bibliothèque NetworkX [Hag08].

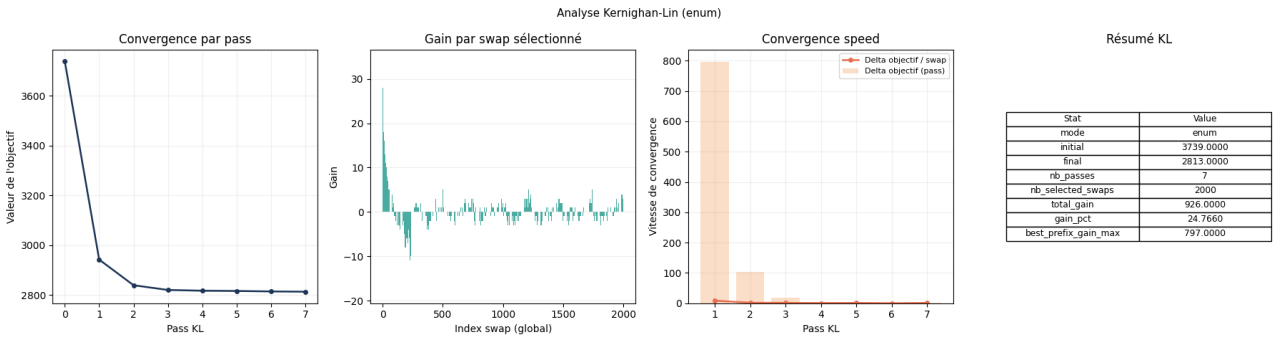
La fonction NetworkX `kernighan_lin_bisection` implémente l'heuristique KL standard pour le partitionnement de graphe bidirectionnel, suivant la stratégie de voisinage basée sur le swap. Pour vérifier l'exactitude de notre implémentation KL, nous avons systématiquement comparé les valeurs objectives obtenues avec celles retournées par l'implémentation de la bibliothèque NetworkX. Afin d'assurer une comparaison équitable et reproductible avec NetworkX, nous avons considéré comme solution initiale comme une solution initiale de partition dans laquelle les sommets d'un indice pair appartiennent à une classe et ceux d'indice impair à l'autre classe.

D'après le tableau 9, nous pouvons voir que les trois implémentations trouvent généralement des valeurs MinCut similaires pour de petits graphes, mais des différences apparaissent à mesure que la taille augmente. Ces différences surviennent parce que KL est une heuristique de recherche locale : l'algorithme explore le voisinage de la partition actuelle pour maximiser le gain, mais il n'est pas garanti qu'il atteigne l'optimum global. Notre version *KL enum* recherche explicitement la meilleure paire à permuter dans chaque itération. Notre version *KL heap* utilise un tas max (max-heap) qui stocke les gains maximaux à la racine. En revanche, l'implémentation de NetworkX peut utiliser des heuristiques supplémentaires ou des stratégies de rupture de lien, entraînant de légères variations dans les valeurs objectives.

En termes de temps de calcul, l'implémentation de la bibliothèque est toujours plus rapide, surtout pour les instances plus grandes.

Ceci est attendu car dans notre version *KL enum*, nous testons explicitement toutes les paires possibles (a, b) entre les deux classes afin de trouver le meilleur échange à chaque étape. Si chaque classe contient environ $n/2$ nœuds, alors, au début, nous évaluons approximativement $(n/2)^2$, qui est $\mathcal{O}(n^2)$, paires possibles. Puisque nous pouvons effectuer jusqu'à $n/2$ swaps dans une itération KL, cette recherche quadratique est répétée environ $\mathcal{O}(n)$ fois. En conséquence, le coût total devient $\mathcal{O}(n) \times \mathcal{O}(n^2) = \mathcal{O}(n^3)$ dans le pire des cas.

Les deux graphes 6 7 montrent que le temps de calcul augmente fortement avec le nombre d'itérations pour $n = 500$, mais la version *KL heap* reste nettement plus rapide que *KL enum*. Cela confirme que l'utilisation d'une structure de données optimisée (heap) permet de réduire significativement le coût de sélection des paires. En revanche, les valeurs de MinCut restent proches, ce qui indique que les deux versions conservent une qualité de solution similaire.



Stat	Value
mode	enum
initial	3739.0000
final	2813.0000
nb_passes	7
nb_selected_swaps	2000
total_gain	926.0000
gain_pct	24.7660
best_prefix_gain_max	797.0000

FIGURE 6 – Analyse du KL enum pour $n = 500$, $p = 2$, $\epsilon = 0$.

Tandis que l'approche théorique KL et éventuellement NetworkX optimisent la sélection de paires avec des structures de données efficaces, réduisant le temps d'exécution pratique, ce qui est de $\mathcal{O}(n^2 \cdot log(n))$. Ainsi, nous avons envisagé de modifier la structure de données afin d'éviter un coût en $\mathcal{O}(n)$ par passe. Avec un max-heap, nous avons pu atteindre un coût en $\mathcal{O}(\log n)$ par passe, ce qui conduit à une complexité totale en $\mathcal{O}(log(n)) \times \mathcal{O}(n^2) = \mathcal{O}(n^2 \cdot log(n))$ dans le pire des cas.

Dans l'ensemble, ces résultats confirment que KL est efficace pour le partitionnement en deux classes et produit rapidement des solutions de bonne qualité pour des graphes de taille petite à moyenne. Toutefois, la valeur du MinCut peut légèrement varier en fonction des choix d'implémentation et de la gestion des égalités. Ainsi, notre implémentation reste moins efficace que celle de la bibliothèque, qui

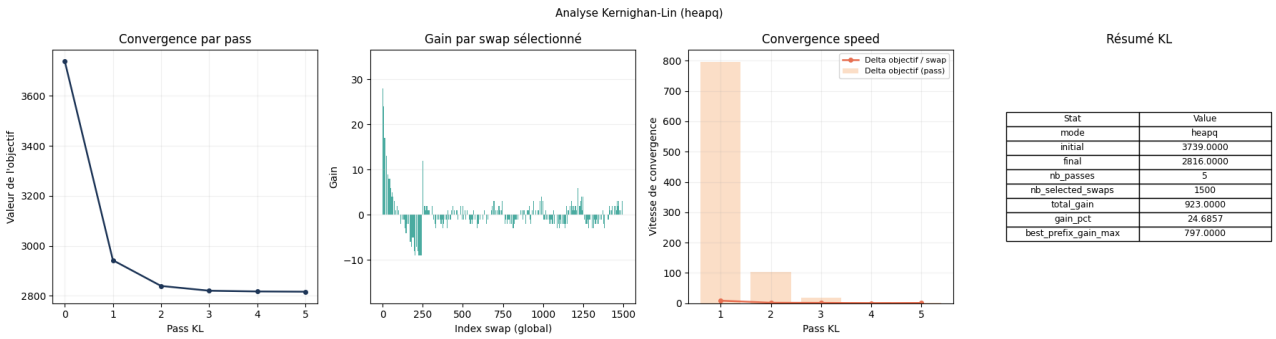


FIGURE 7 – Analyse du KL heap pour $n = 500$, $p = 2$, $\epsilon = 0$.

est optimisée, et pourrait être améliorée en utilisant des structures de données plus adaptées afin de réduire le coût de sélection.

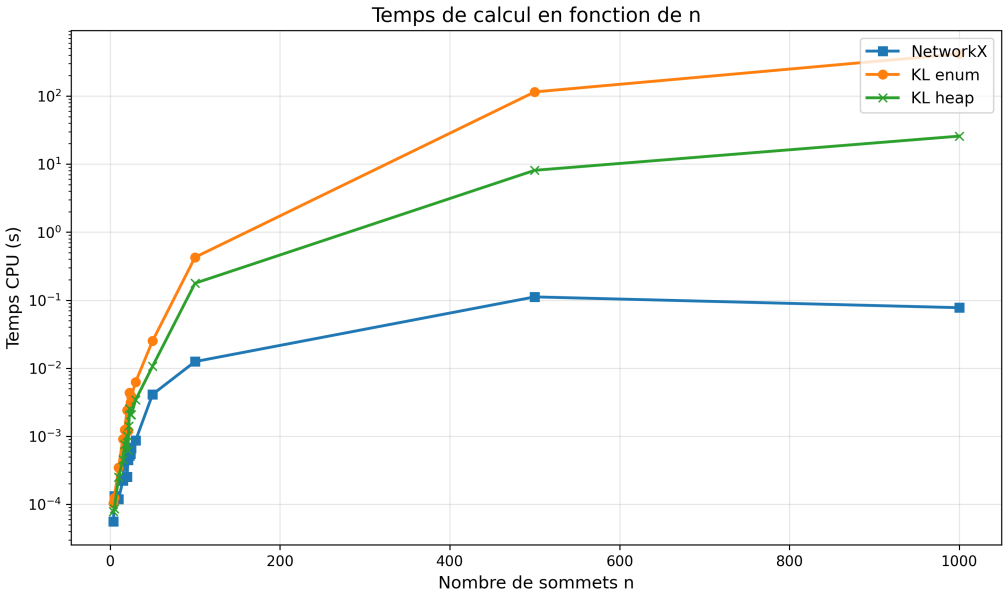


FIGURE 8 – Comparaison des temps de calcul des trois versions de KL

4.2.3 Gradient avec heuristique

Pour pallier la sensibilité de l'algorithme à la solution initiale, comme pour la première version, une stratgéie multi-starts est implémentée. Comme indiqué auparavant cette amélioration est liée étroitement au seuil de solutions améliorantes autorisé. Afin d'évaluer l'impact du paramètre k sur les performances de notre heuristique améliorée nous avons mené une série de tests en faisant varié le paramètre k tout en maintenat le nombre de classes constant. Nous avons également réalisé une seconde série de tests en fixant la valeur de k et en modifiant le nombre de classes. Cette approche permet de comparer les performances de l'heuristique face à des structures de partitions de tailles différentes. Les résultats numériques détaillés sont résumé dans la section A dans les tableaux —.

4.3 Le recuit simulé

Le recuit simulé est un méta-algorithme stochastique. Le fait qu'il soit stochastique signifie qu'il est nécessaire de faire tourner l'algorithme plusieurs fois afin de déterminer la performance générale. L'algorithme explore le voisinage d'une solution en effectuant des déplacements aléatoires. À chaque itération, un sommet est choisi au hasard et déplacé vers une nouvelle classe. Le coût de cette opération dépend du nombre de voisins du sommet dans le graphe et le nombre de classes p. On note deg le degré moyen des sommets. La boucle principale répète N_{iter} fois ces déplacements pour chaque température

T. La température diminue à chaque étape selon le facteur α. Le nombre de paliers est constant pour des valeurs fixées de α et T_{final}. Le paramètre ϵ n'influence pas la complexité au pire cas, mais affecte le nombre de voisins acceptables : une valeur faible de ϵ rend la contrainte d'équilibre plus stricte, ce qui réduit le nombre de mouvements effectifs par palier de température. La valeur de la fonction objectif est mise à jour de manière incrémentale après chaque mouvement. Cette mise à jour locale ne dépend que du degré du sommet modifié, ce qui donne une complexité en $O(\text{deg})$. Grâce à cette approche, on évite de recalculer toute la fonction objectif à chaque itération, ce qui coûterait $O(n+m)$. Cette optimisation réduit fortement le temps de calcul et rend le recuit simulé beaucoup plus efficace. La complexité d'une exécution est donc $O(N_{iter} * (deg + p))$. La méthode MultiStart lance l'algorithme $nb_{runs} = 50$ fois à partir de solutions initiales différentes. Sa complexité totale est donc $O(nb_{runs} * N_{iter} * (deg + p))$. Le facteur N_{iter} dans la boucle interne est le principal facteur limitant pour les grands graphes. Une réduction de ce facteur permettrait d'améliorer les performances de façon significative.

Dans notre projet, nous avons testé toutes les combinaisons de $\alpha \in \{0.85, 0.90, 0.95, 0.99\}$ pour la fonction $g(T)$ et $N_{iter} \in \{n, nlog(n), n^2\}$ avec $nb_{runs} = 50,\ p = 2$ et $\epsilon = 0$ afin de trouver un compromis entre qualité des solutions et temps de calcul. Les autres paramètres ont été fixés ou calculés comme décrit dans la section (5). Les résultats montrent que la valeur de α influence peu la qualité finale.

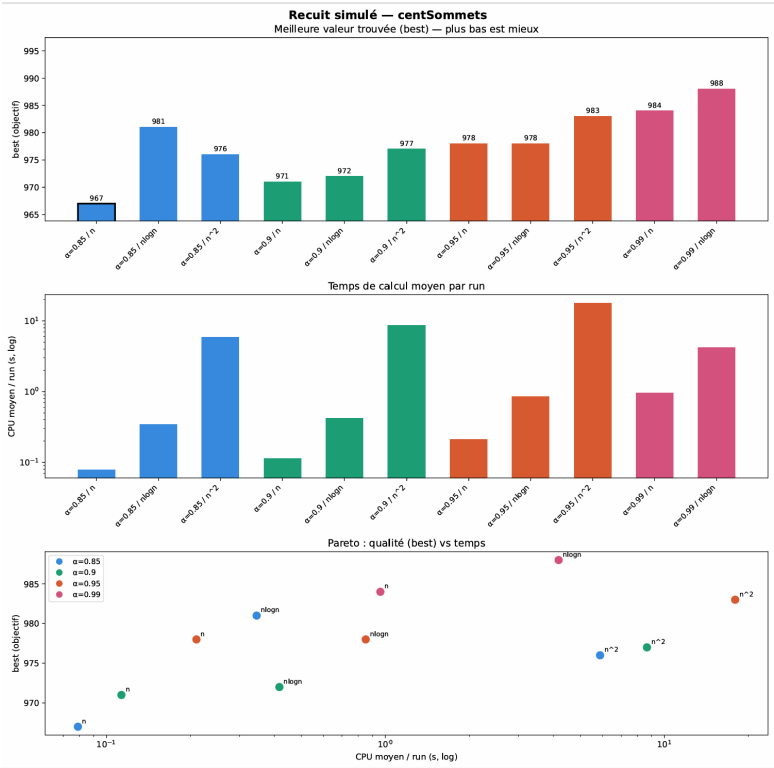


FIGURE 9 – Comparaison des temps de calcul des algorithmes étudiés.

Sur tous les graphes, les valeurs de mean varient très peu entre $\alpha = 0.85$ et $\alpha = 0.99$ (écart $< 1\%$). Par exemple, sur l'instance `centSommets` (voir (9)), la différence entre les meilleures solutions obtenues avec $\alpha = 0.85$ et $\alpha = 0.99$ reste inférieure à $2\,\%$. En revanche, le temps de calcul varie fortement : les configurations avec $\alpha = 0.95$ et $\alpha = 0.99$ sont respectivement environ 2 et 8 fois plus lentes que $\alpha = 0.90$, car un facteur de refroidissement élevé augmente le nombre de paliers de température. À l'inverse, $\alpha = 0.85$ refroidit trop vite : l'algorithme explore moins longtemps et atteint des solutions de moins bonne qualité. Dans l'ensemble, $\alpha = 0.90$ offre le meilleur compromis entre qualité et temps de calcul.

Le nombre d'itérations par palier a un impact encore plus marqué. La configuration $N_{\text{iter}} = n^2$ est systématiquement la plus lente : elle peut être entre 20 et 80 fois plus coûteuse que $N_{\text{iter}} = n$, sans amélioration notable de la valeur *best*. Cela s'explique par le fait qu'un très grand nombre d'itérations à température fixe produit surtout des mouvements redondants. La configuration $N_{\text{iter}} = n \log n$ constitue un compromis intermédiaire, mais elle reste 3 à 4 fois plus lente que $N_{\text{iter}} = n$ pour une qualité similaire. En théorie, on ne choisit presque jamais $N_{\text{iter}} = n$, car ce nombre d'itérations

est souvent considéré comme trop faible pour bien explorer le voisinage. Cependant, dans notre cas particulier, nos tests montrent que cette valeur suffit. Nous retenons donc $N_{\text{iter}} = n$.

Pour la section (5), on fixe $\alpha = 0.9$ et $N_{iter} = n$, car cette configuration offre la meilleure qualité compromis sur les instances testées tout en maintenant un temps de calcul raisonnable. Les résultats numériques détaillés sont résumés dans la section A dans le tableau 10.

La figure 10 illustre le comportement du recuit simulé pour une exécution unique, permettant d'observer en détail sa convergence et la qualité des solutions obtenues.

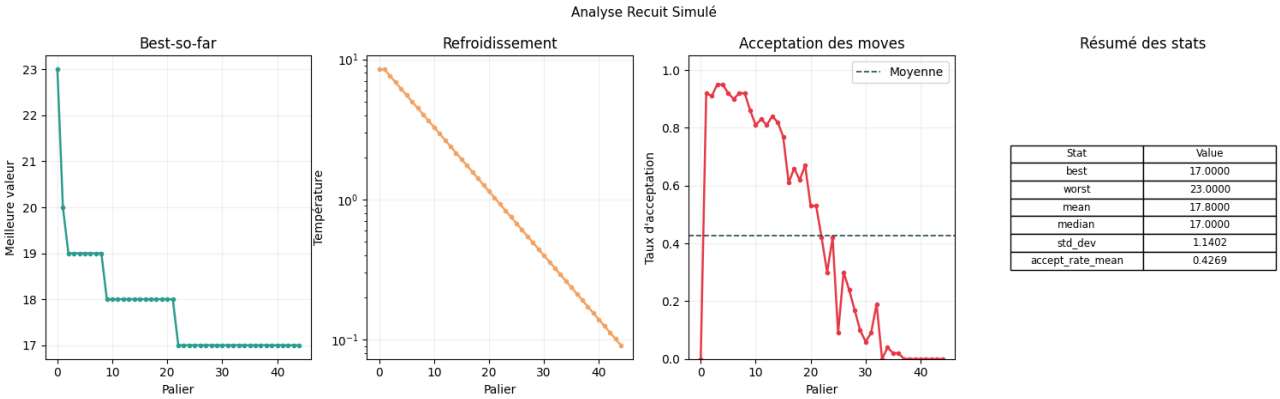
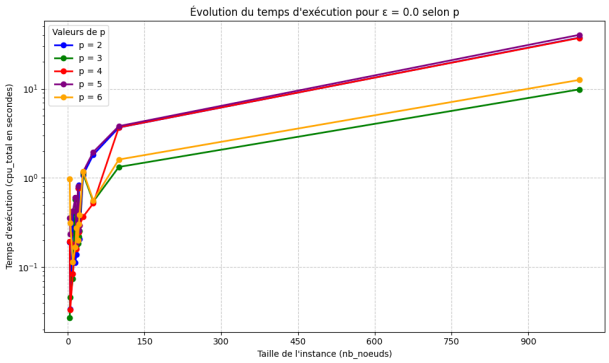
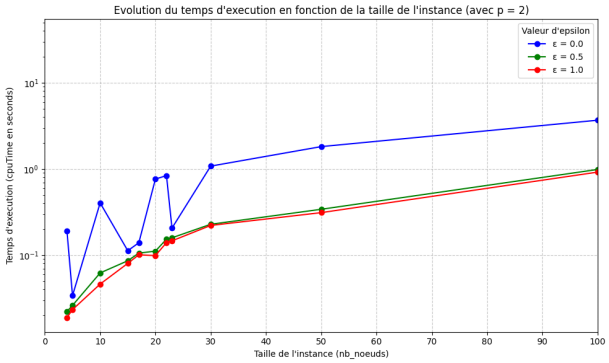


FIGURE 10 – Analyse du recuit simulé pour $n = 10$, $p = 4$, $\epsilon = 0.5$

Les graphiques représentés sur 11 illustrent respectivement l'évolution des temps d'exécution en fonction de la taille des instances pour des classes différentes 11a et pour des valeurs ϵ différentes 11b.



(a) Evolution du temps de calcul en fonction de la taille des instances pour différentes classes (avec $\epsilon = 0$).



(b) Evolution du temps de calcul en fonction de la taille des instances pour différentes valeurs d'ϵ (avec $p = 2$)

FIGURE 11 – Comparaison des temps de calcul selon la taille des instances.

4.4 Algorithme génétique simple (AGS)

4.5 Analyse des résultats de l'AGS

Nous analysons ici l'impact des paramètres p (nombre de classes) et ϵ (contrainte d'équilibre) sur la performance de l'AGS.

La complexité computationnelle de l'AGS dépend principalement de la taille de la population et du nombre de générations. À chaque génération, tous les individus sont évalués, et chaque évaluation nécessite de calculer la fonction objective avec une complexité $O(n+m)$, où n est le nombre de sommets (nœuds) dans le graphe et m est le nombre d'arêtes. Par conséquent, la complexité globale peut être estimée comme suit :

$$O(G \times P \times (n + m)),$$

où G correspond au nombre de générations et P à la taille de la population. Cela explique pourquoi l'algorithme reste efficace pour les instances de taille moyenne mais peine à traiter des graphiques très grands.

Pour POP_SIZE, une population trop faible dégrade la qualité des solutions, tandis qu'une population trop grande augmente fortement le temps d'exécution. La valeur 30 offre un bon compromis entre qualité et coût de calcul.

Pour $GENERATIONS$, l'augmentation du nombre de générations améliore d'abord les résultats, puis les gains deviennent faibles. La valeur 50 permet une convergence satisfaisante avec un temps raisonnable.

Nous avons donc retenu $POP_SIZE = 30$ et $GENERATIONS = 50$ comme compromis entre qualité des solutions et temps de calcul.

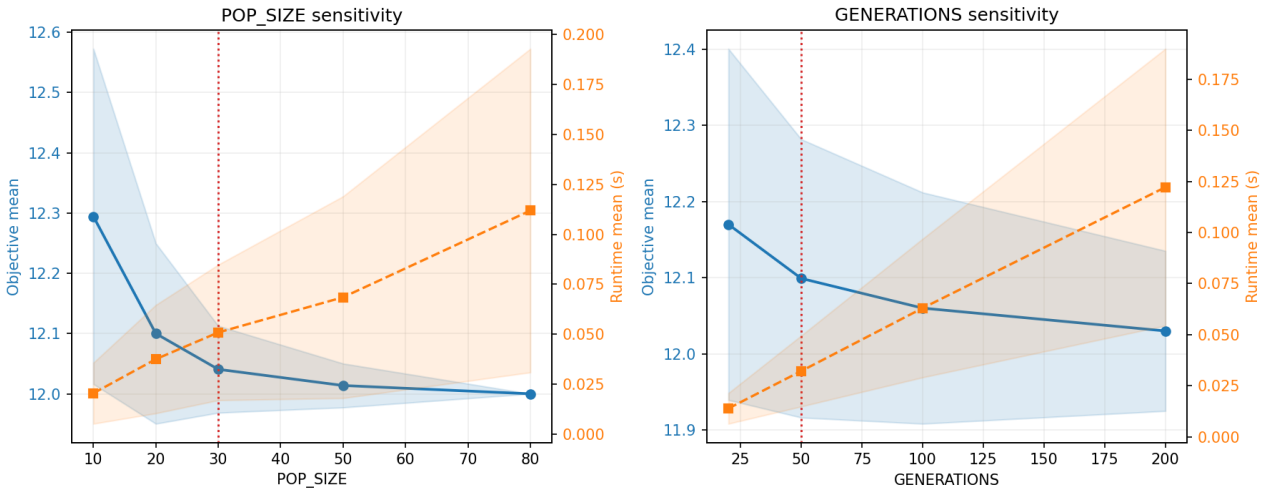


FIGURE 12 – Compromis qualité / temps pour l'instance $n = 10$.

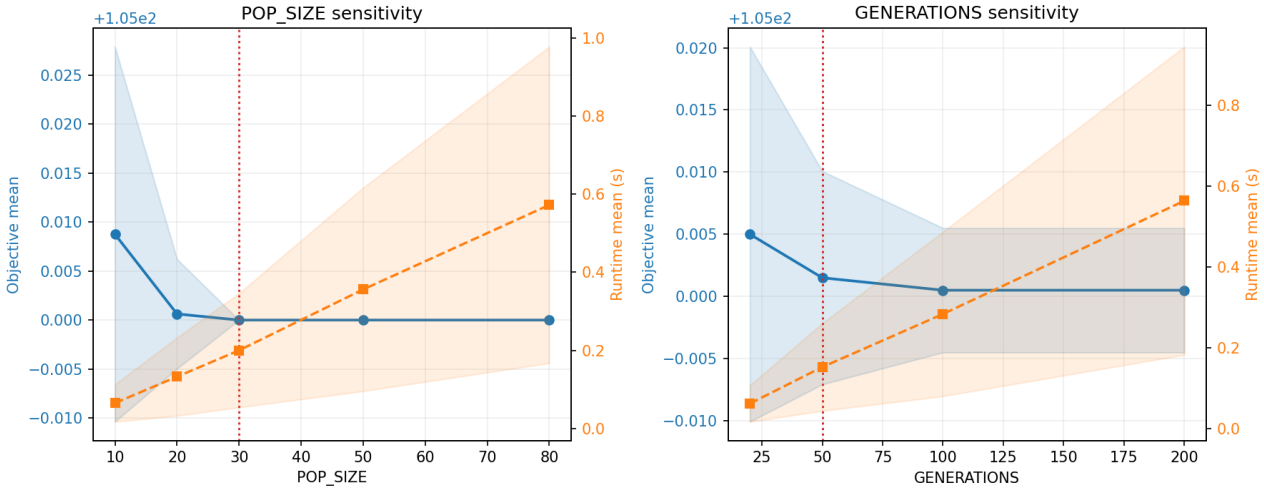


FIGURE 13 – Compromis qualité / temps pour l'instance $n = 21$.

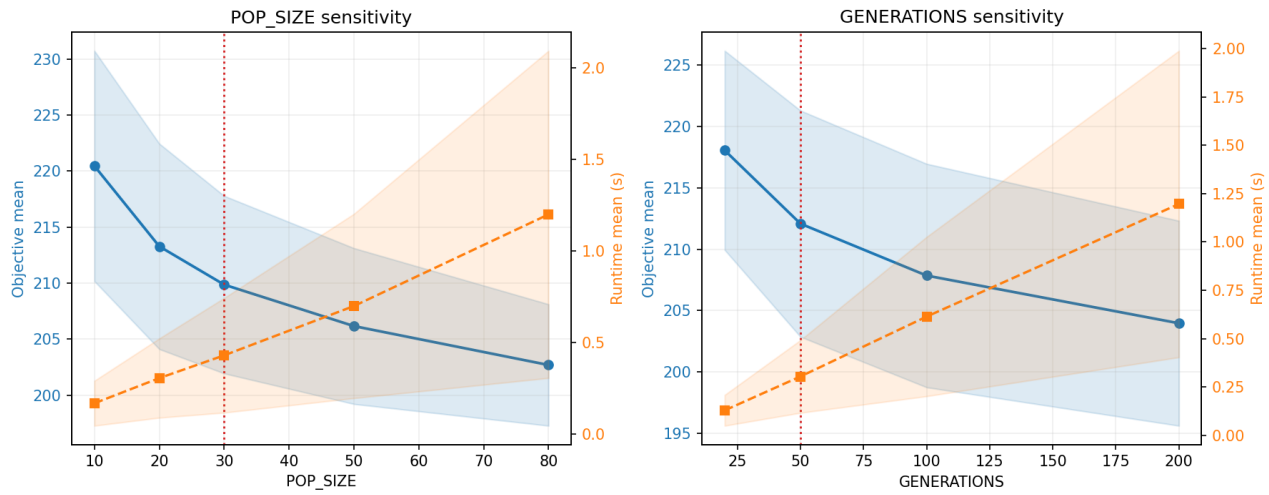


FIGURE 14 – Compromis qualité / temps pour l'instance $n = 50$.

Afin de valider ces résultats théoriques, nous analysons maintenant le comportement expérimental de l'AGS sur une instance représentative.

La figure 15 illustre le comportement de l'AGS pour une exécution unique, permettant d'observer en détail sa convergence et la qualité des solutions obtenues. Afin de reproduire la figure voici les paramètres $n = 10$, $p = 4$, $\epsilon = 0.5$ et une graine aléatoire fixe ($seed = 45$).

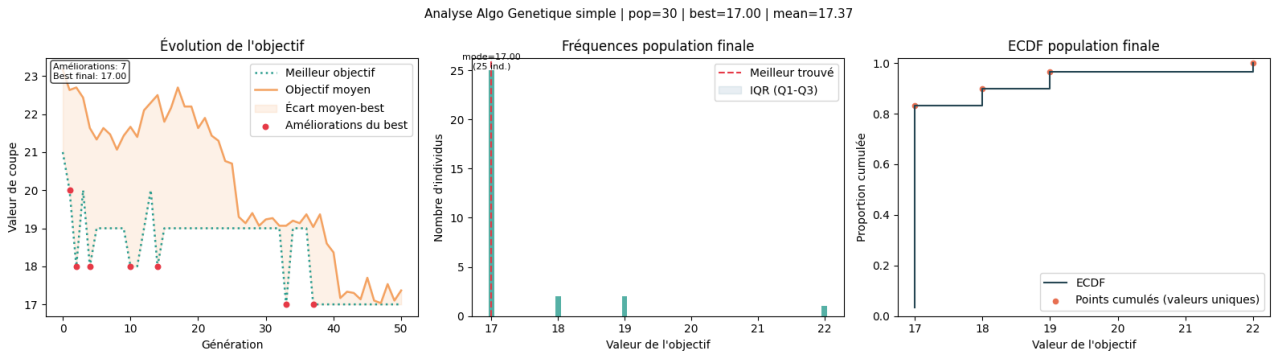


FIGURE 15 – Analyse de l'AGS pour $n = 10$, $p = 4$, $\epsilon = 0.5$, seed $= 45$ (convergence, distribution des solutions et ECDF).

À partir de ces résultats, nous commençons par analyser la courbe de convergence de l'algorithme qui montre l'évolution de la meilleure valeur objective à travers les générations. Nous observons une amélioration rapide au début, suivie d'une phase de stabilisation, ce qui indique que l'algorithme exploite rapidement les régions prometteuses de l'espace de recherche.

Deuxièmement, la distribution des valeurs objectives dans la population finale met en évidence la diversité des solutions. Une distribution plus concentrée autour de valeurs objectives faibles indique une meilleure qualité de convergence, tandis qu'un écart plus large suggère une exploration restante.

Pour quantifier plus formellement cette concentration, on utilise la fonction de répartition empirique (ECDF), définie par :

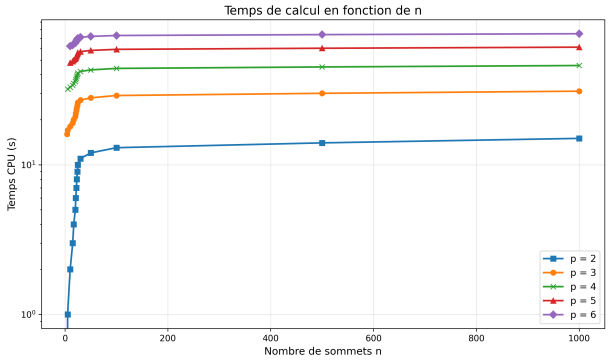
$$\hat{F}(x) = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}(X_i \leqslant x),$$

où X_i sont les valeurs objectives. L'axe des x représente la valeur objective et l'axe des y la proportion d'individus avec une valeur inférieure ou égale à x. Une courbe ECDF plus raide indique que la plupart des individus sont concentrés dans une gamme étroite de solutions de bonne qualité. Ce dernier permet de confirmer l'efficacité du processus de recherche.

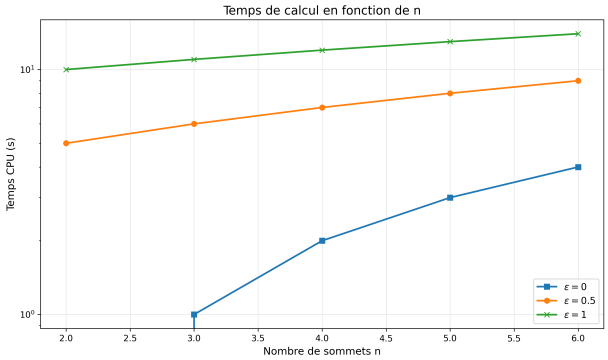
Nous nous intéressons maintenant à l'impact des paramètres sur le temps de calcul de l'AGS. Les figures 16 associées présentent l'évolution du temps de calcul en fonction de la taille du problème et

des paramètres de l'AGS. Globalement, on observe que le temps de calcul augmente lorsque le nombre de classes p augmente. De plus, on observe que le temps de calcul augmente lorsque la taille du graphe n augmente. Cela s'explique par le fait que chaque génération nécessite l'évaluation complète de la population sur toutes les arêtes du graphe.

L'AGS fonctionne bien sur les instances de petite et moyenne taille, fournissant des solutions de bonne qualité grâce à son équilibre entre diversification et intensification. Cependant, ses performances se dégradent sur les instances à grande échelle en raison de son coût computationnel. Les résultats soulignent l'importance du réglage des paramètres, en particulier pour ϵ et p. En conclusion, l'AGS est une métaheuristique flexible et efficace, mais son évolutivité reste une limitation pour les très grands graphes.



(a) Temps de calcul en fonction de n (pour $\epsilon = 0$).



(b) Temps de calcul en fonction de p (pour $n = 10$).

FIGURE 16 – Comparaison des temps de calcul selon le nombre de sommets et le nombre de classes.

5 Analyse globale et discussion

Dans cette section, nous présentons les résultats résumés dans les deux tableaux (3) et (4) et nous les analysons plus en détail quel algorithme donne les meilleurs résultats et lequel est le plus adapté selon la taille des instances. Nous nous restreignons ici au cas $p = 2$ et $\epsilon = 0$, de manière à rendre la comparaison avec l'algorithme KL pertinente et cohérente.

Le *Gap* mesure l'écart relatif entre la solution d'un algorithme et la meilleure solution trouvée sur la même instance. Il est calculé par la formule

$$\text{Gap} = \frac{f_{\text{algo}} - f_{\text{opt}}}{f_{\text{opt}}} \times 100,$$

puis on prend la moyenne sur l'ensemble des instances où l'algorithme n'atteint pas la solution optimale.

TABLE 3 – Comparaison des trois algorithmes pour $p = 2$, $\epsilon = 0$, $n < 30$

Algorithme	#Instances Opt	Gap moyen (%)	Temps CPU moyen (s)
Ènumération	11/11	0.0	73.01
Gradient	9/11	10.69	0.02
KL enum	10/11	2.33	0.00184
KL heap	10/11	2.33	0.00103
Recuit Simule	9/11	11.35	0.37351
AGS	9/11	2.22	0.08165

Le *Win-Rate (%)* correspond à la proportion d'instances où un algorithme obtient la meilleure valeur de la fonction objectif parmi les méthodes comparées. Pour chaque instance, on identifie la solution la plus faible, puis on compte uniquement les cas où un algorithme atteint strictement cette valeur (les égalités ne sont pas comptées).

TABLE 4 – Comparaison Gradient vs KL pour $p = 2$, $\epsilon = 0$ sur toutes des instances

Algorithme	Win-Rate (%)	Temps CPU moyen (s)
Gradient	20	59.7
KL enum	80	258.5
KL heap	80	227.13
Recuit Simule	...	33.39
AGS	(todo@Anny)	222.37

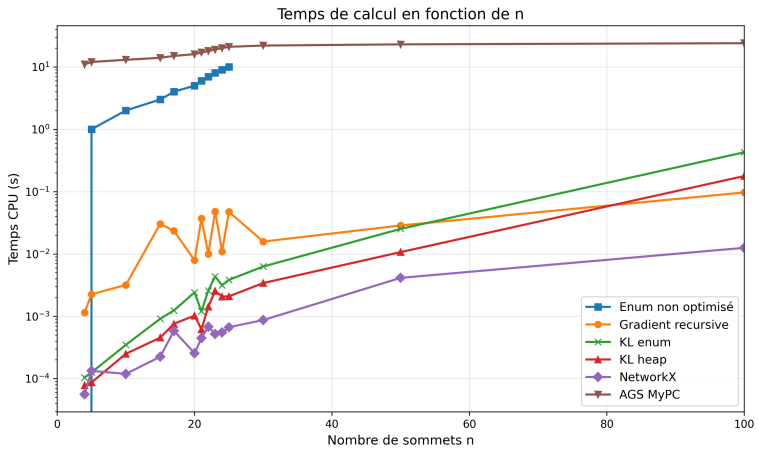


FIGURE 17 – Comparaison des temps de calcul des algorithmes étudiés.

Les résultats des tables 3 et 4, ainsi que le graphique 17, montrent que les différentes méthodes n'ont pas le même comportement selon la taille des instances.

Pour les petits graphes ($n < 30$), les algorithmes atteignent presque toujours la solution optimale. Le Gradient et les deux versions de KL (enum et heap) produisent des résultats très proches. Toutefois, KL reste légèrement supérieur en termes de qualité de solution (gap plus faible) et de temps de calcul, en particulier pour la version optimisée avec heap et l'implémentation de la bibliothèque NetworkX. Le Gradient peut parfois s'éloigner davantage de l'optimum, ce qui explique un gap moyen plus élevé.

Lorsque la taille des instances augmente, les différences deviennent plus marquées. Les méthodes KL apparaissent plus stables : elles trouvent plus souvent la meilleure valeur du MinCut, ce qui se reflète dans sa Win-Rate élevée. À l'inverse, le Gradient dépend beaucoup de la solution initiale et peut facilement rester bloqué dans un minimum local. Cela explique qu'il gagne moins souvent.

En revanche, nos implémentations de KL deviennent très lentes pour les grandes instances et peut même atteindre la limite d'une heure. La version du Gradient impléméntée, au contraire, reste rapide, même si la qualité de sa solution est parfois moins bonne.

En résumé, si l'objectif est d'obtenir la meilleure qualité possible, l'algorithme KL (enum et heap) implementé est le meilleur choix, si l'objectif est d'obtenir une solution rapidement sur un grand graphe, le Gradient est plus adapté.

Enfin, il est important de noter que l'implémentation de la bibliothèque NetworkX surpasse nos versions tant en termes de rapidité que de précision, s'imposant comme la solution de référence pour ces tests.

6 Conclusion

Dans ce projet, nous avons mis en place des méthodes différentes pour résoudre le problème de partition des sommets d'un graphes en p classes peu équitables et une méthode dédiée à un cas exceptionnel de ce dernier (partition en 2 classes peu équitables). Dans un premier temps, nous avons présenté la problématique étudiée, expliqué les méthodes citées. Ensuite nous les avons implémenté en python, afin d'effectuer des expérimentations et différentes analyses. Nous avons analysé chaque algorithme implémenté à part entière en faisant varier les paramètres initiaux de chacun, pour qu'à la fin comparer entre les méthodes simultanément.

A Résultats détaillés pour les algorithmes

A.1 L'algorithme d'énumération explicite

Instance	**p**	**MinCut**			**CPU (s)**		
		$\epsilon = 0.0$	$\epsilon = 0.5$	$\epsilon = 1.0$	$\epsilon = 0.0$	$\epsilon = 0.5$	$\epsilon = 1.0$
quatreSommets	2	2	1	0	8.32e-5	9.42e-5	2.16e-4
cinqSommets	2	4	3	0	2.85e-4	1.56e-4	5.72e-4
dixSommets	2	13	8	0	3.93e-3	6.63e-3	6.66e-3
quinzeSommets	2	43	25	0	2.43e-1	4.36e-1	4.35e-1
dixSeptSommets	2	60	36	0	1.11	2.10	2.12
vingtSommets	2	40	23	0	6.66	15.38	15.47
vingtEtunSommets	2	107	77	0	24.09	50.56	51.84
vingtDeuxSommets	2	103	72	0	34.78	105.30	105.91
vingtTroisSommets	2	116	68	0	101.85	227.90	226.47
vingtQuatreSommets	2	131	89	0	152.88	485.89	490.48
vingtCinqSommets	2	152	110	0	481.49	1108.22	1099.68
quatreSommets	3	2	2	1	2.46e-4	4.56e-4	2.85e-4
cinqSommets	3	6	4	3	6.82e-4	8.24e-4	9.02e-4
dixSommets	3	17	13	11	2.56e-1	3.32e-1	3.84e-1
quinzeSommets	3	61	43	37	61.12	176.09	186.00
dixSeptSommets	3	81	60	45	732.04	1926.59	2073.98
vingtSommets	3	–	–	–	–	TimeLimit	–
quatreSommets	4	4	2	2	5.62e-4	7.18e-4	9.50e-4
cinqSommets	4	6	6	4	3.04e-3	3.03e-3	3.47e-3
dixSommets	4	21	17	13	3.85	5.68	6.52
quinzeSommets	4	–	–	–	–	TimeLimit	–
cinqSommets	5	8	6	6	6.78e-3	9.86e-3	9.87e-3
dixSommets	5	25	21	17	28.54	43.54	58.72
quinzeSommets	5	–	–	–	–	TimeLimit	–
dixSommets	6	25	21	17	189.94	308.80	375.49
quinzeSommets	6	–	–	–	–	TimeLimit	–

Table 5 – Résultats détaillés pour l'algorithme d'énumération explicite.

Instance	p	$\epsilon = 0.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	2	2	14	3	2.72	3	0.453	0.001
cinqSommets	2	4	50	4	4	4	0	0.002
dixSommets	2	13	2	21	16.44	17	1.667	0.003
quinzeSommets	2	43	11	48	44.54	44	1.281	0.03
dixSeptSommets	2	60	18	63	61	61	0.903	0.023
vingtSommets	2	47	3	60	52.04	52	2.927	0.007
vingtEtunSommets	2	107	30	110	107.62	107	0.90	0.037
vingtDeuxSommets	2	107	2	116	111.64	111.5	2.183	0.009
vingtTroisSommets	2	116	5	120	117.72	117.5	1.088	0.047
vingtQuatreSommets	2	131	11	133	131.96	132	0.637	0.010
vingtCinqSommets	2	152	15	156	153.26	153	1.174	0.047
trenteSommets	2	199	1	212	204.48	204.5	2.682	0.015
cinquanteSommets	2	241	2	277	254.6	254	8.621	0.028
centSommets	2	960	1	1040	1007.14	1009.5	20.659	0.096
cinqCentSommets	2	3651	1	3839	3756.34	3752.5	40.847	1.163
milleSommets	2	4864	1	5097	5004.7	5006.5	54.79	4.365
dixMilleSommets	2	4997574	1	5003373	5000498.68	5000332.0	1444.479	1008.975

(a) Avec $\epsilon = 0.0$

Instance	p	$\epsilon = 0.5$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	2	1.0	30	2.0	1.4	1.0	0.494	0.0017
cinqSommets	2	3.0	47	4.0	3.06	3.0	0.239	0.0022
dixSommets	2	8.0	24	13.0	8.96	9.0	1.159	0.0105
quinzeSommets	2	25.0	18	33.0	28.02	27.0	2.979	0.0428
dixSeptSommets	2	36.0	6	48.0	41.76	45.0	4.582	0.0581
vingtSommets	2	23.0	1	43.0	32.8	32.0	4.384	0.0643
vingtEtunSommets	2	77.0	17	80.0	78.04	78.0	0.9467	0.1235
vingtDeuxSommets	2	73.0	6	82.0	75.72	75.0	2.0408	0.1328
vingtTroisSommets	2	68.0	2	84.0	76.88	81.0	6.206	0.2156
vingtQuatreSommets	2	89.0	6	102.0	94.78	91.0	5.7045	0.1915
vingtCinqSommets	2	110.0	17	114.0	111.62	112.0	1.426	0.2300
trenteSommets	2	133.0	4	145.0	139.62	140.0	3.463	0.4356
cinquanteSommets	2	138.0	1	176.0	155.3	156.0	7.451	1.5976
centSommets	2	631.0	1	700.0	670.66	671.5	13.997	24.1713
cinqCentSommets	2	2201.0	1	2420.0	2311.94	2320.5	41.496	2519.277
milleSommets	2	4864	1	5097	5004.7	5006.5	54.79	4.365
dixMilleSommets	2	4997574	1	5003373	5000498.68	5000332.0	1444.479	1008.975

(b) Avec $\epsilon = 0.5$

Instance	p	$\epsilon = 1.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	2	0.0	50	0.0	0.0	0.0	0.0	0.001894
cinqSommets	2	0.0	50	0.0	0.0	0.0	0.0	0.003186
dixSommets	2	0.0	49	12.0	0.24	0.0	1.697056	0.016232
quinzeSommets	2	0.0	50	0.0	0.0	0.0	0.0	0.081903
dixSeptSommets	2	0.0	50	0.0	0.0	0.0	0.0	0.105654
vingtSommets	2	0.0	50	0.0	0.0	0.0	0.0	0.136391
vingtEtunSommets	2	0.0	50	0.0	0.0	0.0	0.0	0.260213
vingtDeuxSommets	2	0.0	50	0.0	0.0	0.0	0.0	0.292287
vingtTroisSommets	2	0.0	50	0.0	0.0	0.0	0.0	0.348386
vingtQuatreSommets	2	0.0	50	0.0	0.0	0.0	0.0	0.404068
vingtCinqSommets	2	0.0	50	0.0	0.0	0.0	0.0	0.464462
trenteSommets	2	0.0	50	0.0	0.0	0.0	0.0	0.91745
cinquanteSommets	2	0.0	50	0.0	0.0	0.0	0.0	3.0842
centSommets	2	0.0	50	0.0	0.0	0.0	0.0	47.0922
cinqCentSommets	2	-	-	-	-	-	-	TimeLimit
milleSommets	2	-	-	-	-	-	-	TimeLimit
dixMilleSommets	2	-	-	-	-	-	-	TimeLimit

(c) Avec $\epsilon = 1.0$

TABLE 6 – Résultats détaillés pour l'algorithme du gradient selon différentes valeurs d'équitabilité (ϵ)

Instance	p	$\epsilon = 0.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	4	4.0	50	4.0	4.0	4.0	0.0	0.001852
cinqSommets	4	6.0	50	6.0	6.0	6.0	0.0	0.005129
dixSommets	4	21.0	13	25.0	22.28	22.0	1.088718	0.029633
quinzeSommets	4	70.0	27	72.0	70.54	70.0	0.645550	0.047690
dixSeptSommets	4	87.0	4	93.0	89.94	90.0	1.742740	0.051823
vingtSommets	4	72.0	1	86.0	79.3	79.0	3.065609	0.014012
vingtEtunSommets	4	159.0	28	161.0	159.52	159.0	0.646498	0.073945
vingtDeuxSommets	4	162.0	1	169.0	164.58	164.0	1.539812	0.087418
vingtTroisSommets	4	176.0	8	179.0	177.28	177.0	0.833973	0.073151
vingtQuatreSommets	4	197.0	7	199.0	198.16	198.0	0.650275	0.022256
vingtCinqSommets	4	227.0	21	229.0	227.8	228.0	0.782461	0.102855
trenteSommets	4	297.0	6	308.0	300.22	300.0	2.493174	0.349101
cinquanteSommets	4	311.0	1	346.0	330.06	331.0	7.707431	3.734086
centSommets	4	1475.0	1	1543.0	1511.04	1512.5	14.582573	0.201311
cinqCentSommets	4	5551.0	1	5738.0	5624.18	5620.5	41.366796	3.159419
milleSommets	4	7429.0	1	7579.0	7500.34	7496.5	35.247006	11.265790
dixMilleSommets	4	7497515.0	1	7503616.0	7500703.98	7500674.0	1215.762841	1822.879267

(a) Avec $\epsilon = 0.0$

Instance	p	$\epsilon = \frac{1}{2}$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	4	2.0	24	3.0	2.52	3.0	0.504672	0.003285
cinqSommets	4	6.0	50	6.0	6.0	6.0	0.0	0.004482
dixSommets	4	17.0	7	22.0	19.0	19.0	1.324803	0.030656
quinzeSommets	4	58.0	4	65.0	60.92	61.0	1.454620	0.092741
dixSeptSommets	4	74.0	3	80.0	77.34	77.0	1.985766	0.147264
vingtSommets	4	53.0	1	66.0	59.18	59.0	2.854928	0.236543
vingtEtunSommets	4	141.0	24	143.0	141.68	142.0	0.740656	0.263314
vingtDeuxSommets	4	136.0	1	143.0	139.9	140.0	1.693324	0.361403
vingtTroisSommets	4	149.0	5	156.0	151.9	151.0	1.971739	0.403931
vingtQuatreSommets	4	170.0	8	175.0	171.9	171.0	1.488048	0.398679
vingtCinqSommets	4	196.0	13	199.0	197.16	197.0	0.865672	0.555412
trenteSommets	4	251.0	1	263.0	258.46	259.0	2.516963	1.086684
cinquanteSommets	4	268.0	2	316.0	296.16	297.5	10.272531	5.631363
centSommets	4	1198.0	2	1268.0	1227.98	1231.5	18.597224	78.171339
cinqCentSommets	4	-	-	-	-	-	-	TimeLimit
milleSommets	4	-	-	-	-	-	-	TimeLimit
dixMilleSommets	4	-	-	-	-	-	-	TimeLimit

(b) Avec $\epsilon = \frac{1}{2}$

Instance	p	$\epsilon = 1.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	4	2.0	24	3.0	2.52	3.0	0.504672	0.003554
cinqSommets	4	4.0	46	6.0	4.16	4.0	0.548095	0.007035
dixSommets	4	13.0	9	19.0	14.88	15.0	1.364865	0.052365
quinzeSommets	4	43.0	8	49.0	44.92	45.0	1.306811	0.223803
dixSeptSommets	4	60.0	12	63.0	61.22	61.0	0.864020	0.268304
vingtSommets	4	40.0	1	54.0	45.74	45.5	2.947223	0.343194
vingtEtunSommets	4	107.0	27	110.0	107.64	107.0	0.851409	0.655648
vingtDeuxSommets	4	106.0	1	115.0	110.04	110.0	2.059919	0.677246
vingtTroisSommets	4	116.0	10	120.0	117.58	118.0	1.108225	0.807316
vingtQuatreSommets	4	131.0	19	133.0	131.68	132.0	0.586933	0.885575
vingtCinqSommets	4	152.0	16	156.0	153.06	153.0	1.038248	1.240609
trenteSommets	4	195.0	1	208.0	201.64	202.0	3.141298	2.119478
cinquanteSommets	4	207.0	2	283.0	226.42	226.0	11.962680	9.196384
centSommets	4	897.0	1	966.0	935.16	937.5	17.034611	139.505830
cinqCentSommets	4	-	-	-	-	-	-	TimeLimit
milleSommets	4	-	-	-	-	-	-	TimeLimit
dixMilleSommets	4	-	-	-	-	-	-	TimeLimit

(c) Avec $\epsilon = 1.0$

TABLE 7 – Résultats détaillés pour l'algorithme du gradient selon différentes valeurs d'équitabilité (ϵ)

Instance	p	$\epsilon = 0.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	5	4.0	50	4.0	4.0	4.0	0.0	0.001505
cinqSommets	5	8.0	50	8.0	8.0	8.0	0.0	0.002977
dixSommets	5	25.0	7	29.0	26.48	26.0	0.973946	0.006856
quinzeSommets	5	76.0	3	80.0	78.12	78.0	0.961292	0.013246
dixSeptSommets	5	94.0	5	99.0	96.66	97.0	1.408574	0.062046
vingtSommets	5	80.0	3	89.0	84.34	84.5	2.300044	0.018754
vingtEtunSommets	5	167.0	29	169.0	167.52	167.0	0.677330	0.143370
vingtDeuxSommets	5	172.0	6	177.0	174.22	174.0	1.313820	0.104497
vingtTroisSommets	5	186.0	7	191.0	188.12	188.0	1.479658	0.110919
vingtQuatreSommets	5	209.0	9	211.0	209.86	210.0	0.452205	0.101998
vingtCinqSommets	5	246.0	18	249.0	246.78	247.0	0.736511	0.030060
trenteSommets	5	324.0	6	332.0	326.78	326.0	2.121705	0.040163
cinquanteSommets	5	395.0	1	425.0	408.36	408.0	6.989088	0.074795
centSommets	5	1580.0	1	1647.0	1618.44	1620.0	16.061311	0.261048
cinqCentSommets	5	5935.0	1	6068.0	6003.56	6007.0	33.354105	4.122997
milleSommets	5	7888.0	1	8098.0	8009.58	8012.0	39.935198	15.179119
dixMilleSommets	5	7998146.0	1	8002986.0	8000663.88	8000697.0	1247.885585	2407.708230

(a) Avec $\epsilon = 0.0$

Instance	p	$\epsilon = \frac{1}{2}$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	5	2.0	2	4.0	3.36	3.0	0.562792	0.001403
cinqSommets	5	6.0	50	6.0	6.0	6.0	0.0	0.007262
dixSommets	5	21.0	15	25.0	22.36	22.0	1.156349	0.029423
quinzeSommets	5	62.0	3	67.0	64.28	64.0	1.088718	0.138267
dixSeptSommets	5	81.0	14	84.0	81.98	82.0	0.769044	0.253303
vingtSommets	5	62.0	1	76.0	67.86	67.0	3.416706	0.278326
vingtEtunSommets	5	144.0	23	147.0	144.68	145.0	0.740656	0.552562
vingtDeuxSommets	5	151.0	3	156.0	153.24	153.0	1.333401	0.503208
vingtTroisSommets	5	164.0	1	172.0	168.54	169.5	2.383703	0.449755
vingtQuatreSommets	5	175.0	20	177.0	175.74	176.0	0.694292	0.801912
vingtCinqSommets	5	212.0	21	214.0	212.8	213.0	0.782461	0.888425
trenteSommets	5	287.0	3	295.0	291.04	291.0	2.231134	1.272595
cinquanteSommets	5	312.0	1	361.0	335.12	335.0	11.358068	6.580416
centSommets	5	1326.0	1	1407.0	1372.1	1371.5	20.513062	99.950695
cinqCentSommets	5	-	-	-	-	-	-	TimeLimit
milleSommets	5	-	-	-	-	-	-	TimeLimit
dixMilleSommets	5	-	-	-	-	-	-	TimeLimit

(b) Avec $\epsilon = \frac{1}{2}$

Instance	p	$\epsilon = 1.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	5	2.0	2	4.0	3.32	3.0	0.551066	0.001413
cinqSommets	5	6.0	50	6.0	6.0	6.0	0.0	0.007000
dixSommets	5	17.0	4	23.0	19.1	19.0	1.359021	0.047261
quinzeSommets	5	58.0	2	64.0	60.68	61.0	1.406109	0.167293
dixSeptSommets	5	74.0	9	80.0	76.26	76.0	1.882356	0.266853
vingtSommets	5	52.0	2	65.0	57.86	58.0	3.263965	0.411793
vingtEtunSommets	5	132.0	23	134.0	132.64	133.0	0.662709	0.676350
vingtDeuxSommets	5	136.0	4	143.0	139.16	139.0	1.730400	0.745231
vingtTroisSommets	5	136.0	2	146.0	140.16	139.0	3.328050	0.988572
vingtQuatreSommets	5	159.0	6	167.0	161.92	160.0	2.805534	1.024034
vingtCinqSommets	5	196.0	12	199.0	197.02	197.0	0.742033	1.165672
trenteSommets	5	253.0	2	262.0	258.12	259.0	2.395915	2.249146
cinquanteSommets	5	273.0	1	306.0	289.4	289.5	7.695745	10.180042
centSommets	5	1156.0	1	1232.0	1190.6	1190.0	15.824290	162.654079
cinqCentSommets	5	-	-	-	-	-	-	TimeLimit
milleSommets	5	-	-	-	-	-	-	TimeLimit
dixMilleSommets	5	-	-	-	-	-	-	TimeLimit

(c) $\epsilon = 1.0$

TABLE 8 – Résultats détaillés pour l'algorithme du gradient selon différentes valeurs d'équitabilité (ϵ)

Instance	**KL enum**		**KL heap**		**NetworkX/KL**	
	MinCut	**CPU (s)**	**MinCut**	**CPU (s)**	**MinCut**	**CPU (s)**
quatreSommets	2	$1.04e{-}4$	2	$1.23e{-}4$	2	$5.60e{-}5$
cinqSommets	4	$1.25e{-}4$	4	$7.08e{-}5$	4	$1.33e{-}4$
dixSommets	13	$3.47e{-}4$	13	$2.20e{-}4$	13	$1.19e{-}4$
quinzeSommets	**44**	$9.10e{-}4$	44	$1.02e{-}3$	43	$2.24e{-}4$
dixSeptSommets	60	$1.24e{-}3$	60	$1.45e{-}3$	60	$5.85e{-}4$
vingtSommets	40	$2.42e{-}3$	40	$1.76e{-}3$	41	$2.55e{-}4$
vingtEtunSommets	107	$1.20e{-}3$	107	$7.13e{-}4$	107	$4.46e{-}4$
vingtDeuxSommets	103	$2.59e{-}3$	103	$1.36e{-}3$	103	$6.82e{-}4$
vingtTroisSommets	116	$4.34e{-}3$	116	$2.42e{-}3$	117	$5.21e{-}4$
vingtQuatreSommets	131	$3.14e{-}3$	131	$2.12e{-}3$	131	$5.50e{-}4$
vingtCinqSommets	152	$3.83e{-}3$	152	$2.37e{-}3$	152	$6.67e{-}4$
trenteSommets	189	$6.28e{-}3$	189	$3.47e{-}3$	190	$8.69e{-}4$
cinquanteSommets	200	$2.52e{-}2$	200	$1.21e{-}2$	197	$4.13e{-}3$
centSommets	846	$4.27e{-}1$	846	$1.71e{-}1$	839	$1.25e{-}2$
cinqCentSommets	2813	$1.15e2$	2816	$1.08e1$	2843	$1.12e{-}1$
milleSommets	3458	$4.16e2$	3442	$4.07e1$	3469	$7.76e{-}2$
dixMilleSommets	–	TimeLimit	–	TimeLimit	4856485	$2.39e2$

TABLE 9 – Comparaison entre notre implémentation de KL et la version de la bibliothèque
Les valeurs en gras correspondent aux solutions non optimales (i.e. différentes de la valeur optimale de la fonction).

Instance	**p**	$\epsilon = 0.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	2	**2**	16	3	2.68	3	0.471	0.190
cinqSommets	2	**4**	50	4	4.00	4	0.000	0.034
dixSommets	2	**13**	1	20	16.54	17	1.656	0.405
quinzeSommets	2	**43**	44	44	43.12	43	0.328	0.115
dixSeptSommets	2	**60**	50	60	60.00	60	0.000	0.142
vingtSommets	2	47	3	59	53.02	54	2.945	0.766
vingtEtunSommets	2	**107**	50	107	107.00	107	0.000	0.233
vingtDeuxSommets	2	108	2	115	111.18	111	2.007	0.842
vingtTroisSommets	2	**116**	50	116	116.00	116	0.000	0.210
vingtQuatreSommets	2	**131**	12	133	131.96	132	0.669	0.897
vingtCinqSommets	2	**152**	50	152	152.00	152	0.000	0.303
trenteSommets	2	197	1	212	204.86	205	3.024	1.085
cinquanteSommets	2	229	1	273	256.58	257.5	8.790	1.826
centSommets	2	959	1	1054	1011.40	1010	17.411	3.705
cinqCentSommets	2	3646	1	3860	3759.34	3754	44.347	18.582
milleSommets	2	4825	1	5089	4996.78	5002.5	53.369	37.641
dixMilleSommets	2	4997754	1	5003514	5000710.22	5000659.5	1391.738	733.517

(a) Avec $\epsilon = 0.0$

Instance	**p**	$\epsilon = 0.5$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	2	**1**	50	1	1.00	1	0.000	0.023
cinqSommets	2	**3**	50	3	3.00	3	0.000	0.027
dixSommets	2	**8**	44	9	8.12	8	0.328	0.064
quinzeSommets	2	**25**	26	31	25.80	25	1.125	0.088
dixSeptSommets	2	**36**	1	40	37.70	38	0.814	0.108
vingtSommets	2	**23**	6	34	27.60	28	3.003	0.114
vingtEtunSommets	2	**77**	28	79	77.46	77	0.542	0.149
vingtDeuxSommets	2	**72**	1	77	74.64	74.5	1.102	0.157
vingtTroisSommets	2	69	3	73	70.54	71	0.762	0.163
vingtQuatreSommets	2	**89**	17	91	89.70	90	0.544	0.180
vingtCinqSommets	2	**110**	28	112	110.62	110	0.780	0.195
trenteSommets	2	135	2	147	140.72	141	2.595	0.234
cinquanteSommets	2	139	1	169	154.82	154	6.577	0.348
centSommets	2	677	1	717	695.42	696	10.408	1.011
cinqCentSommets	2	2379	1	2537	2454.52	2458	35.627	4.598
milleSommets	2	2994	1	3141	3069.70	3066.5	37.206	7.625
dixMilleSommets	2	-	-	-	-	-	-	TimeLimit

(b) Avec $\epsilon = 0.5$

Instance	**p**	$\epsilon = 1.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	2	**0**	50	0	0.00	0	0.000	0.020
cinqSommets	2	**0**	50	0	0.00	0	0.000	0.024
dixSommets	2	**0**	50	0	0.00	0	0.000	0.047
quinzeSommets	2	**0**	50	0	0.00	0	0.000	0.082
dixSeptSommets	2	**0**	50	0	0.00	0	0.000	0.104
vingtSommets	2	**0**	50	0	0.00	0	0.000	0.101
vingtEtunSommets	2	**0**	50	0	0.00	0	0.000	0.137
vingtDeuxSommets	2	**0**	50	0	0.00	0	0.000	0.143
vingtTroisSommets	2	**0**	50	0	0.00	0	0.000	0.150
vingtQuatreSommets	2	**0**	50	0	0.00	0	0.000	0.159
vingtCinqSommets	2	**0**	50	0	0.00	0	0.000	0.174
trenteSommets	2	**0**	50	0	0.00	0	0.000	0.227
cinquanteSommets	2	**0**	50	0	0.00	0	0.000	0.319
centSommets	2	**0**	50	0	0.00	0	0.000	0.943
cinqCentSommets	2	**0**	50	0	0.00	0	0.000	4.202
milleSommets	2	**0**	50	0	0.00	0	0.000	7.060
dixMilleSommets	2	-	-	-	-	-	-	TimeLimit

(c) Avec $\epsilon = 1.0$

TABLE 10 – Résultats détaillés pour l'algorithme du recuit simulé selon différentes valeurs d'équitabilité (ϵ) pour p=2.

Instance	**p**	$\epsilon = 0.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	3	**2**	50	2	2.00	2	0.000	0.028
cinqSommets	3	**6**	50	6	6.00	6	0.000	0.046
dixSommets	3	**17**	35	18	17.30	17	0.463	0.075
quinzeSommets	3	62	1	67	65.00	65	1.195	0.581
dixSeptSommets	3	**81**	50	81	81.00	81	0.000	0.175
vingtSommets	3	55	26	57	55.60	55	0.700	0.186
vingtEtunSommets	3	144	20	147	144.78	145	0.764	0.811
vingtDeuxSommets	3	142	47	143	142.06	142	0.240	0.222
vingtTroisSommets	3	156	50	156	156.00	156	0.000	0.257
vingtQuatreSommets	3	175	13	177	175.94	176	0.682	0.911
vingtCinqSommets	3	203	50	203	203.00	203	0.000	0.273
trenteSommets	3	267	1	278	272.28	272	2.792	1.126
cinquanteSommets	3	268	1	278	272.56	272	2.492	0.551
centSommets	3	1133	1	1167	1151.90	1151	7.391	1.353
cinqCentSommets	3	3871	3	3955	3902.58	3902.5	18.933	6.787
milleSommets	3	4756	2	4840	4793.86	4792	22.094	10.005
dixMilleSommets	3	-	-	-	-	-	-	TimeLimit

(a) Avec $\epsilon = 0.0$

Instance	**p**	$\epsilon = 0.5$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	3	**2**	50	2	2.00	2	0.000	0.044
cinqSommets	3	**4**	50	4	4.00	4	0.000	0.029
dixSommets	3	**13**	20	16	13.92	14	0.877	0.076
quinzeSommets	3	**43**	27	46	43.78	43	0.975	0.096
dixSeptSommets	3	**60**	44	61	60.12	60	0.328	0.117
vingtSommets	3	41	5	50	44.62	45	2.364	0.132
vingtEtunSommets	3	107	50	107	107.00	107	0.000	0.182
vingtDeuxSommets	3	105	1	114	108.94	109	2.045	0.178
vingtTroisSommets	3	116	48	117	116.04	116	0.198	0.176
vingtQuatreSommets	3	131	49	132	131.02	131	0.141	0.203
vingtCinqSommets	3	152	50	152	152.00	152	0.000	0.233
trenteSommets	3	195	3	208	200.04	200	2.878	0.273
cinquanteSommets	3	200	1	235	219.68	219.5	7.965	0.393
centSommets	3	877	1	971	924.90	925.5	22.194	1.124
cinqCentSommets	3	2956	1	3254	3100.96	3104.5	62.094	4.974
milleSommets	3	3601	1	4002	3814.82	3816	83.198	8.376
dixMilleSommets	3	-	-	-	-	-	-	TimeLimit

(b) Avec $\epsilon = 0.5$

Instance	**p**	$\epsilon = 1.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	3	**1**	50	1	1.00	1	0.000	0.037
cinqSommets	3	**3**	50	3	3.00	3	0.000	0.026
dixSommets	3	**11**	27	13	11.48	11	0.544	0.052
quinzeSommets	3	**37**	2	43	40.18	40	1.466	0.091
dixSeptSommets	3	**45**	4	49	47.04	47	1.049	0.107
vingtSommets	3	29	4	41	34.60	35	3.149	0.111
vingtEtunSommets	3	95	11	98	96.04	96	0.727	0.150
vingtDeuxSommets	3	91	5	98	94.16	94	1.963	0.153
vingtTroisSommets	3	92	1	97	94.40	94	0.904	0.162
vingtQuatreSommets	3	111	9	113	112.18	112	0.720	0.174
vingtCinqSommets	3	132	8	136	133.44	133	0.951	0.190
trenteSommets	3	171	1	186	178.60	179	3.175	0.241
cinquanteSommets	3	172	1	215	193.98	195	8.592	0.350
centSommets	3	794	1	874	832.38	834	17.128	1.023
cinqCentSommets	3	2846	1	2999	2935.06	2937.5	32.256	4.538
milleSommets	3	3561	1	3793	3665.12	3659.5	43.144	7.577
dixMilleSommets	3	-	-	-	-	-	-	TimeLimit

(c) Avec $\epsilon = 1.0$

TABLE 11 – Résultats détaillés pour l'algorithme du recuit simulé pour p=3 et différentes valeurs d'équiprobabilité (ϵ)

Instance	p	$\epsilon = 0.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	4	**4**	50	4	4.00	4	0.000	0.195
cinqSommets	4	**6**	50	6	6.00	6	0.000	0.034
dixSommets	4	**21**	50	21	21.00	21	0.000	0.086
quinzeSommets	4	70	50	70	70.00	70	0.000	0.184
dixSeptSommets	4	87	21	88	87.58	88	0.499	0.163
vingtSommets	4	73	1	85	78.80	79	2.507	0.772
vingtEtunSommets	4	159	50	159	159.00	159	0.000	0.227
vingtDeuxSommets	4	162	50	162	162.00	162	0.000	0.248
vingtTroisSommets	4	176	50	176	176.00	176	0.000	0.295
vingtQuatreSommets	4	197	13	199	197.86	198	0.606	0.918
vingtCinqSommets	4	227	50	227	227.00	227	0.000	0.291
trenteSommets	4	296	48	297	296.04	296	0.198	0.372
cinquanteSommets	4	306	2	323	314.72	315	4.291	0.527
centSommets	4	1487	1	1538	1514.38	1517	12.744	3.719
cinqCentSommets	4	5559	1	5692	5630.62	5628	33.385	19.277
milleSommets	4	7431	1	7596	7513.10	7519.5	41.485	37.391
dixMilleSommets	4	7498345	1	7503167	7500434.74	7500411	1165.744	736.123

(a) Avec $\epsilon = 0.0$

Instance	p	$\epsilon = 0.5$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	4	**2**	50	2	2.00	2	0.000	0.024
cinqSommets	4	**6**	50	6	6.00	6	0.000	0.034
dixSommets	4	**17**	27	18	17.46	17	0.503	0.062
quinzeSommets	4	58	9	61	59.24	59	0.771	0.101
dixSeptSommets	4	74	3	76	75.18	75	0.523	0.120
vingtSommets	4	52	2	59	55.50	55.5	1.669	0.122
vingtEtunSommets	4	141	47	142	141.06	141	0.240	0.170
vingtDeuxSommets	4	135	2	141	137.92	138	1.469	0.168
vingtTroisSommets	4	149	11	152	150.14	150	0.857	0.183
vingtQuatreSommets	4	170	29	171	170.42	170	0.499	0.194
vingtCinqSommets	4	196	37	198	196.28	196	0.497	0.209
trenteSommets	4	250	1	261	255.78	256	2.297	0.257
cinquanteSommets	4	264	1	298	282.02	283	7.694	0.399
centSommets	4	1178	1	1237	1202.92	1202.5	13.206	1.101
cinqCentSommets	4	4056	1	4276	4140.24	4135.5	39.476	4.878
milleSommets	4	4909	1	5226	5099.26	5101.5	52.910	8.233
dixMilleSommets	4	-	-	-	-	-	-	TimeLimit

(b) Avec $\epsilon = 0.5$

Instance	p	$\epsilon = 1.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
quatreSommets	4	**2**	50	2	2.00	2	0.000	0.024
cinqSommets	4	**4**	50	4	4.00	4	0.000	0.043
dixSommets	4	**13**	17	16	14.10	14	1.035	0.056
quinzeSommets	4	43	14	47	44.30	44	1.216	0.090
dixSeptSommets	4	60	35	61	60.30	60	0.463	0.108
vingtSommets	4	41	2	50	45.00	45	2.347	0.117
vingtEtunSommets	4	107	49	108	107.02	107	0.141	0.158
vingtDeuxSommets	4	106	4	112	109.18	109	1.687	0.165
vingtTroisSommets	4	116	42	117	116.16	116	0.370	0.168
vingtQuatreSommets	4	131	46	132	131.08	131	0.274	0.182
vingtCinqSommets	4	152	49	153	152.02	152	0.141	0.199
trenteSommets	4	193	2	207	199.50	199	3.334	0.251
cinquanteSommets	4	201	1	235	220.40	219	7.491	0.367
centSommets	4	859	1	953	914.58	915	20.326	1.032
cinqCentSommets	4	2982	1	3194	3094.34	3095	51.566	4.673
milleSommets	4	3672	1	3974	3775.48	3774	69.076	7.737
dixMilleSommets	4	-	-	-	-	-	-	TimeLimit

(c) Avec $\epsilon = 1.0$

TABLE 12 – Résultats détaillés pour l'algorithme du recuit simulé pour p=4 et différentes valeurs d'équiprobabilité (ϵ)

Instance	p	$\epsilon = 0.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
cinqSommets	5	**8**	50	8	8.00	8	0.000	0.237
dixSommets	5	**25**	6	28	26.46	27	0.788	0.427
quinzeSommets	5	76	8	79	77.64	78	0.964	0.603
dixSeptSommets	5	94	44	96	94.16	94	0.468	0.187
vingtSommets	5	80	3	90	84.40	84	2.515	0.807
vingtEtunSommets	5	167	50	167	167.00	167	0.000	0.242
vingtDeuxSommets	5	172	47	173	172.06	172	0.240	0.263
vingtTroisSommets	5	186	45	187	186.10	186	0.303	0.298
vingtQuatreSommets	5	209	50	209	209.00	209	0.000	0.359
vingtCinqSommets	5	246	23	248	246.62	247	0.635	0.983
trenteSommets	5	322	1	334	326.90	326.5	2.358	1.174
cinquanteSommets	5	397	3	430	408.84	410	6.001	1.962
centSommets	5	1581	1	1653	1618.08	1619	15.854	3.839
cinqCentSommets	5	5923	1	6065	6013.10	6014.5	30.953	20.047
milleSommets	5	7903	1	8110	8011.86	8009.5	48.061	40.466
dixMilleSommets	5	7997149	1	8003707	8000457.66	8000405.5	1102.697	752.311

(a) Avec $\epsilon = 0.0$

Instance	p	$\epsilon = 0.5$						
		best	nb_f	worst	mean	med	sd	CPU(s)
cinqSommets	5	**6**	50	6	6.00	6	0.000	0.032
dixSommets	5	**21**	48	22	21.04	21	0.198	0.086
quinzeSommets	5	61	6	65	62.86	63	1.069	0.108
dixSeptSommets	5	81	44	82	81.12	81	0.328	0.117
vingtSommets	5	62	3	68	64.54	64.5	1.417	0.136
vingtEtunSommets	5	144	32	146	144.46	144	0.676	0.172
vingtDeuxSommets	5	150	4	156	152.34	153	1.379	0.186
vingtTroisSommets	5	164	2	169	165.52	165	0.974	0.197
vingtQuatreSommets	5	175	38	176	175.24	175	0.431	0.201
vingtCinqSommets	5	212	26	214	212.54	212	0.613	0.221
trenteSommets	5	286	2	296	289.68	290	1.911	0.279
cinquanteSommets	5	312	1	337	326.02	327	6.222	0.407
centSommets	5	1335	1	1403	1370.52	1373.5	16.522	1.144
cinqCentSommets	5	4674	1	4874	4794.00	4789	41.704	5.151
milleSommets	5	5857	1	6205	6017.30	6017	65.100	8.553
dixMilleSommets	5	-	-	-	-	-	-	TimeLimit

(b) Avec $\epsilon = 0.5$

Instance	p	$\epsilon = 1.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
cinqSommets	5	**6**	50	6	6.00	6	0.000	0.031
dixSommets	5	**17**	16	19	17.78	18	0.616	0.057
quinzeSommets	5	58	7	61	59.38	59	0.878	0.096
dixSeptSommets	5	74	6	77	75.36	75	0.851	0.116
vingtSommets	5	52	1	61	56.64	56	2.328	0.121
vingtEtunSommets	5	132	21	134	132.68	133	0.653	0.163
vingtDeuxSommets	5	136	5	141	138.32	139	1.236	0.165
vingtTroisSommets	5	136	4	140	138.08	138	1.007	0.171
vingtQuatreSommets	5	159	5	161	160.02	160	0.473	0.183
vingtCinqSommets	5	196	28	199	196.58	196	0.758	0.204
trenteSommets	5	252	1	263	257.12	257	2.553	0.253
cinquanteSommets	5	278	1	305	291.14	291	6.405	0.372
centSommets	5	1182	1	1251	1215.46	1213	16.344	1.060
cinqCentSommets	5	4179	1	4481	4275.90	4277.5	54.778	4.965
milleSommets	5	5146	1	5620	5359.96	5364	103.760	7.943
dixMilleSommets	5	-	-	-	-	-	-	TimeLimit

(c) Avec $\epsilon = 1.0$

TABLE 13 – Résultats détaillés pour l'algorithme du recuit simulé pour p=5 et différentes valeurs d'équitabilité (ϵ).

Instance	**p**	$\epsilon = 0.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
dixSommets	6	**25**	50	25	25.00	25	0.000	0.115
quinzeSommets	6	76	41	78	76.20	76	0.452	0.169
dixSeptSommets	6	102	50	102	102.00	102	0.000	0.279
vingtSommets	6	71	1	78	74.12	74	1.452	0.203
vingtEtunSommets	6	177	50	177	177.00	177	0.000	0.277
vingtDeuxSommets	6	182	50	182	182.00	182	0.000	0.309
vingtTroisSommets	6	196	50	196	196.00	196	0.000	0.391
vingtQuatreSommets	6	219	4	221	220.04	220	0.450	0.979
vingtCinqSommets	6	246	43	247	246.14	246	0.351	0.320
trenteSommets	6	337	3	346	340.50	341	1.787	1.193
cinquanteSommets	6	356	1	373	365.66	366	3.679	0.570
centSommets	6	1488	3	1519	1499.96	1499	7.626	1.636
cinqCentSommets	6	5112	1	5181	5143.66	5143	14.351	6.349
milleSommets	6	6359	1	6463	6411.50	6410.5	22.592	12.740
dixMilleSommets	6	-	-	-	-	-	-	TimeLimit

(a) Avec $\epsilon = 0.0$

Instance	**p**	$\epsilon = 0.5$						
		best	nb_f	worst	mean	med	sd	CPU(s)
dixSommets	6	**21**	43	22	21.14	21	0.351	0.063
quinzeSommets	6	70	48	72	70.06	70	0.314	0.110
dixSeptSommets	6	87	4	90	88.12	88	0.594	0.122
vingtSommets	6	65	2	73	68.76	69	1.721	0.143
vingtEtunSommets	6	159	49	160	159.02	159	0.141	0.171
vingtDeuxSommets	6	162	42	163	162.16	162	0.370	0.181
vingtTroisSommets	6	176	41	177	176.18	176	0.388	0.194
vingtQuatreSommets	6	197	31	198	197.38	197	0.490	0.219
vingtCinqSommets	6	227	42	228	227.16	227	0.370	0.219
trenteSommets	6	296	19	298	296.84	297	0.766	0.274
cinquanteSommets	6	311	1	331	321.92	321	4.664	0.398
centSommets	6	1358	1	1449	1402.60	1403.5	21.298	1.216
cinqCentSommets	6	4659	1	4947	4788.00	4783.5	67.095	5.215
milleSommets	6	5797	1	6111	5946.42	5939	71.546	8.506
dixMilleSommets	6	-	-	-	-	-	-	TimeLimit

(b) Avec $\epsilon = 0.5$

Instance	**p**	$\epsilon = 1.0$						
		best	nb_f	worst	mean	med	sd	CPU(s)
dixSommets	6	**17**	10	19	18.04	18	0.669	0.058
quinzeSommets	6	61	4	66	63.12	63	1.118	0.101
dixSeptSommets	6	81	23	82	81.54	82	0.503	0.114
vingtSommets	6	55	2	63	57.86	58	1.818	0.121
vingtEtunSommets	6	144	36	147	144.36	144	0.663	0.167
vingtDeuxSommets	6	142	31	144	142.44	142	0.611	0.164
vingtTroisSommets	6	156	30	158	156.42	156	0.538	0.176
vingtQuatreSommets	6	175	29	176	175.42	175	0.499	0.195
vingtCinqSommets	6	203	45	204	203.10	203	0.303	0.202
trenteSommets	6	261	1	275	267.44	267	3.052	0.260
cinquanteSommets	6	273	1	320	284.54	284	7.600	0.370
centSommets	6	1155	1	1215	1179.30	1179.5	11.599	1.074
cinqCentSommets	6	4021	1	4312	4084.06	4079.5	43.272	4.779
milleSommets	6	4972	1	5261	5069.88	5054	63.328	7.833
dixMilleSommets	6	-	-	-	-	-	-	TimeLimit

(c) Avec $\epsilon = 1.0$

TABLE 14 – Résultats détaillés pour l'algorithme du recuit simulé pour p=6 et différentes valeurs d'équitabilité (ϵ).

Instance	p	MinCut			CPU (s)		
		$\epsilon = 0.0$	$\epsilon = 0.5$	$\epsilon = 1.0$	$\epsilon = 0.0$	$\epsilon = 0.5$	$\epsilon = 1.0$
quatreSommets	2	2	1	0	$1.32e{-2}$	$1.56e{-2}$	$1.22e{-2}$
cinqSommets	2	4	3	0	$1.66e{-2}$	$1.60e{-2}$	$1.49e{-2}$
dixSommets	2	13	8	0	$2.56e{-2}$	$2.53e{-2}$	$2.35e{-2}$
quinzeSommets	2	43	25	**5**	$5.14e{-2}$	$4.79e{-2}$	$4.29e{-2}$
dixSeptSommets	2	60	36	0	$5.97e{-2}$	$5.79e{-2}$	$4.95e{-2}$
vingtSommets	2	**41**	**29**	**7**	$5.52e{-2}$	$5.08e{-2}$	$4.64e{-2}$
vingtEtunSommets	2	107	77	0	$9.52e{-2}$	$8.62e{-2}$	$7.62e{-2}$
vingtDeuxSommets	2	**105**	**74**	0	$9.62e{-2}$	$9.04e{-2}$	$7.89e{-2}$
vingtTroisSommets	2	116	**69**	0	$1.06e{-1}$	$9.58e{-2}$	$8.41e{-2}$
vingtQuatreSommets	2	131	89	0	$1.81e{-1}$	$9.97e{-2}$	$9.10e{-2}$
vingtCinqSommets	2	152	110	0	$1.98e{-1}$	$1.22e{-1}$	$1.03e{-1}$
trenteSommets	2	193	135	0	$1.84e{-1}$	$1.51e{-1}$	$1.39e{-1}$
cinquanteSommets	2	209	160	82	$2.61e{-1}$	$1.98e{-1}$	$1.93e{-1}$
centSommets	2	945	701	468	$8.60e{-1}$	$7.31e{-1}$	$6.99e{-1}$
cinqCentSommets	2	3632	3492	3381	$4.40e0$	$3.23e0$	$3.32e0$
milleSommets	2	4854	4755	4716	$6.66e0$	$5.64e0$	$4.78e0$
dixMilleSommets	2	–	–	–	TimeLimit	TimeLimit	TimeLimit
quatreSommets	3	2	2	1	$1.39e{-2}$	$1.36e{-2}$	$1.59e{-2}$
cinqSommets	3	6	4	3	$1.98e{-2}$	$1.68e{-2}$	$1.90e{-2}$
dixSommets	3	17	13	11	$2.51e{-2}$	$2.60e{-2}$	$2.44e{-2}$
quinzeSommets	3	61	**46**	**38**	$5.74e{-2}$	$4.79e{-2}$	$4.76e{-2}$
dixSeptSommets	3	81	60	**46**	$6.41e{-2}$	$6.12e{-2}$	$5.82e{-2}$
vingtSommets	3	60	49	28	$8.69e{-2}$	$5.09e{-2}$	$5.41e{-2}$
vingtEtunSommets	3	144	107	101	$1.32e{-1}$	$9.47e{-2}$	$9.58e{-2}$
vingtDeuxSommets	3	142	106	96	$9.99e{-2}$	$9.62e{-2}$	$9.10e{-2}$
vingtTroisSommets	3	156	117	93	$1.08e{-1}$	$1.03e{-1}$	$9.91e{-2}$
vingtQuatreSommets	3	175	131	111	$1.65e{-1}$	$1.17e{-1}$	$1.08e{-1}$
vingtCinqSommets	3	203	152	139	$1.49e{-1}$	$1.28e{-1}$	$1.21e{-1}$
trenteSommets	3	261	215	181	$1.87e{-1}$	$1.70e{-1}$	$1.69e{-1}$
cinquanteSommets	3	303	276	201	$2.85e{-1}$	$2.18e{-1}$	$2.07e{-1}$
centSommets	3	1253	1140	1095	$8.16e{-1}$	$7.62e{-1}$	$7.58e{-1}$
cinqCentSommets	3	4865	4725	4712	$5.33e0$	$3.54e0$	$3.37e0$
milleSommets	3	6496	6439	6421	$7.92e0$	$5.77e0$	$4.76e0$
dixMilleSommets	3	–	–	–	TimeLimit	TimeLimit	TimeLimit
quatreSommets	4	-	-	-	-	-	-
cinqSommets	4	6	6	4	$2.31e{-2}$	$1.92e{-2}$	$1.59e{-2}$
dixSommets	4	21	18	15	$2.69e{-2}$	$2.55e{-2}$	$2.60e{-2}$
quinzeSommets	4	70	58	43	$5.81e{-2}$	$5.30e{-2}$	$4.92e{-2}$
dixSeptSommets	4	88	74	60	$7.56e{-2}$	$6.18e{-2}$	$5.73e{-2}$
vingtSommets	4	70	54	55	$7.05e{-2}$	$5.55e{-2}$	$5.15e{-2}$
vingtEtunSommets	4	159	141	116	$1.14e{-1}$	$9.54e{-2}$	$9.12e{-2}$
vingtDeuxSommets	4	162	137	105	$9.94e{-2}$	$9.61e{-2}$	$9.54e{-2}$
vingtTroisSommets	4	176	149	117	$1.52e{-1}$	$1.00e{-1}$	$1.04e{-1}$
vingtQuatreSommets	4	197	170	142	$1.62e{-1}$	$1.22e{-1}$	$1.10e{-1}$
vingtCinqSommets	4	227	196	152	$1.76e{-1}$	$1.27e{-1}$	$1.26e{-1}$
trenteSommets	4	296	258	218	$2.15e{-1}$	$1.67e{-1}$	$1.66e{-1}$
cinquanteSommets	4	347	313	276	$2.87e{-1}$	$2.34e{-1}$	$2.32e{-1}$
centSommets	4	1460	1303	1289	$8.90e{-1}$	$8.54e{-1}$	$8.14e{-1}$
cinqCentSommets	4	5495	5379	5350	$5.31e0$	$3.37e0$	$3.38e0$
milleSommets	4	7377	7245	7272	$7.57e0$	$4.76e0$	$5.45e0$
dixMilleSommets	4	–	–	–	TimeLimit	TimeLimit	TimeLimit

TABLE 15 – Résultats détaillés pour AGs.

Les valeurs en gras correspondent aux solutions non optimales (i.e. différentes de la valeur optimale de la fonction).

Instance	**p**	**MinCut**			**CPU (s)**		
		$\epsilon = 0.0$	$\epsilon = 0.5$	$\epsilon = 1.0$	$\epsilon = 0.0$	$\epsilon = 0.5$	$\epsilon = 1.0$
cinqSommets	5	–	–	–	–	–	–
dixSommets	5	25	21	18	$3.99e{-2}$	$2.79e{-2}$	$2.58e{-2}$
quinzeSommets	5	76	61	60	$6.09e{-2}$	$5.43e{-2}$	$5.27e{-2}$
dixSeptSommets	5	94	81	80	$8.35e{-2}$	$6.27e{-2}$	$6.29e{-2}$
vingtSommets	5	73	67	53	$6.15e{-2}$	$5.52e{-2}$	$5.25e{-2}$
vingtEtunSommets	5	167	154	134	$1.23e{-1}$	$9.90e{-2}$	$9.49e{-2}$
vingtDeuxSommets	5	172	150	139	$1.14e{-1}$	$9.29e{-2}$	$9.59e{-2}$
vingtTroisSommets	5	186	170	137	$1.37e{-1}$	$1.05e{-1}$	$1.04e{-1}$
vingtQuatreSommets	5	209	176	168	$1.24e{-1}$	$1.17e{-1}$	$1.25e{-1}$
vingtCinqSommets	5	246	212	200	$2.04e{-1}$	$1.99e{-1}$	$1.27e{-1}$
trenteSommets	5	320	290	254	$2.38e{-1}$	$1.68e{-1}$	$1.64e{-1}$
cinquanteSommets	5	386	350	340	$3.09e{-1}$	$2.34e{-1}$	$2.51e{-1}$
centSommets	5	1569	1480	1379	$1.22e0$	$8.18e{-1}$	$7.94e{-1}$
cinqCentSommets	5	5902	5755	5771	$4.73e0$	$3.71e0$	$3.72e0$
milleSommets	5	7874	7758	7724	$7.15e0$	$5.14e0$	$6.16e0$
dixMilleSommets	5	–	–	–	TimeLimit	TimeLimit	TimeLimit
dixSommets	6	25	**22**	**18**	$3.89e{-2}$	$2.67e{-2}$	$2.51e{-2}$
quinzeSommets	6	76	72	65	$5.60e{-2}$	$5.43e{-2}$	$5.12e{-2}$
dixSeptSommets	6	102	88	85	$7.67e{-2}$	$6.37e{-2}$	$6.14e{-2}$
vingtSommets	6	76	70	60	$7.45e{-2}$	$5.65e{-2}$	$5.19e{-2}$
vingtEtunSommets	6	177	159	144	$1.17e{-1}$	$1.49e{-1}$	$1.30e{-1}$
vingtDeuxSommets	6	182	165	142	$1.45e{-1}$	$9.84e{-2}$	$9.63e{-2}$
vingtTroisSommets	6	196	177	162	$1.65e{-1}$	$1.18e{-1}$	$1.02e{-1}$
vingtQuatreSommets	6	219	198	182	$1.39e{-1}$	$1.18e{-1}$	$1.56e{-1}$
vingtCinqSommets	6	246	227	214	$1.77e{-1}$	$1.32e{-1}$	$1.28e{-1}$
trenteSommets	6	335	296	277	$1.98e{-1}$	$1.74e{-1}$	$2.63e{-1}$
cinquanteSommets	6	400	363	356	$2.48e{-1}$	$2.14e{-1}$	$2.13e{-1}$
centSommets	6	1626	1539	1451	$9.69e{-1}$	$8.06e{-1}$	$9.09e{-1}$
cinqCentSommets	6	6124	6032	6030	$4.34e0$	$3.52e0$	$3.69e0$
milleSommets	6	8181	8075	8115	$8.23e0$	$5.09e0$	$5.00e0$
dixMilleSommets	5	–	–	–	TimeLimit	TimeLimit	TimeLimit

TABLE 16 – Résultats détaillés pour AGs.
Les valeurs en gras correspondent aux solutions non optimales (i.e. différentes de la valeur optimale de la fonction).

Références

[AMO93] R. K. Ahuja, T. L. Magnanti et J. B. Orlin. *Network Flows : Theory, Algorithms, and Applications*. Upper Saddle River, NJ : Prentice Hall, 1993. isbn : 978-0-13-617549-0.

[AR04] K. Andreev et H. Räcke. « Balanced Graph Partitioning ». In : *Proceedings of the Sixteenth Annual ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 2004, p. 120-124. doi : 10.1145/1007912.1007934.

[Cor26] M. M. Corsini. *Fiche sur la Méthode du Gradient et l'Algorithme de Kernighan–Lin*. Fiche de cours. Document PDF disponible sur Moodle. Moodle / Université de Boredaux, 2026. url : https://moodle.u-bordeaux.fr/course/view.php?id=2022.

[Dat23] DataCamp. *Law of Large Numbers : Weak vs Strong*. https://www.datacamp.com/fr/tutorial/law-of-large-numbers. Accessed : 2026-03-04. 2023.

[Hag08] Hagberg, Aric and Schult, Daniel and Swart, Pieter. *NetworkX : Network Analysis in Python*. https://networkx.org/. 2008.

[KGV83] S. Kirkpatrick, C. D. Gelatt et M. P. Vecchi. « Optimization by Simulated Annealing ». In : *Science* 220.4598 (mai 1983), p. 671-680.

[Met+53] N. Metropolis et al. « Equation of State Calculations by Fast Computing Machines ». In : *Journal of Chemical Physics* 21.6 (juin 1953), p. 1087-1092.

[Wik26a] Wikipedia contributors. *Genetic algorithm*. https://simple.wikipedia.org/wiki/Genetic_algorithm. 2026.

[Wik26b] Wikipedia contributors. *Kernighan–Lin algorithm*. https://en.wikipedia.org/wiki/Kernighan-Lin_algorithm. 2026.